DATEXT Diagnostics

Technisches Handbuch

Version 0.99.12.11
Stand: November 2026

Vollständige Dokumentation aller integrierten Features, technischen Hintergründe und Implementierungsdetails.

© 2026 DATEXT GmbH

Inhaltsverzeichnis

Inhaltsverzeichnis 2
1. Projektübersicht 5
2. Technische Grundlagen 6
Plattform & Framework 6
Build & Veröffentlichung 6
Assembly-Informationen 6
3. Architektur & Projektstruktur 7
Ordnerstruktur 7
Architekturprinzip 7
Navigation 7
Seitenleiste & zentrales Style-Wörterbuch 8
4. Navigationsgruppe: System 9
4.1 Übersicht (SystemOverviewDialog) 9
4.2 System-Health (SystemHealthDialog) 9
4.3 UEFI-Boot-Zertifikat (UefiBootCertDialog) 10
4.4 Autorun-Analyse (AutorunAnalysisDialog) 10
4.5 Ereignislog-Viewer (EventLogDialog) 11
4.6 Installierte Software (SystemSoftwareDialog) 13
4.7 Winget-Pakete (WingetDialog) 13
4.8 System-Verwaltung (SystemServicesDialog) 14
4.9 Troubleshooting (TroubleshootingDialog) 15
5. Navigationsgruppe: Netzwerk 17
5.1 Geräte-Scan (DeviceScanDialog) 17
5.2 Ping & Latenz (PingDialog) 19
5.3 Traceroute (TracerouteDialog) 19
5.4 IP & DNS Analyse (DnsLookupDialog) 20
5.5 Port-Scan (PortScanDialog) 21
5.6 Netzwerk & Verbindungen (NetzwerkHubDialog) 21
5.7 DHCP-Discovery (DhcpDiscoveryDialog) 24
5.8 Firewall-Log-Auswertung (FirewallLogDialog) 26
6. Navigationsgruppe: Server & Dienste 28
6.1 AD-Suche (ActiveDirectoryDialog) 28
6.2 Gruppenrichtlinien (GpoAnalysisDialog) 28
6.3 MS-SQL-Analyse (SqlAnalysisDialog) 29
6.4 IIS-Analyse (IisAnalysisDialog) 29
6.5 Hyper-V-Analyse (HyperVDialog) 30
6.6 AD-Sicherheit & Zustand (AdHealthDialog) 31

© 2026 DATEXT

6.7 DNS-Server-Analyse (DnsServerDialog) ... 32
6.8 DHCP-Server-Analyse (DhcpServerDialog) ... 32
6.9 Terminalserver-Analyse (RdsDialog) ... 33
7. Navigationsgruppe: Packet Capture ... 35
7.1 PKTMON Capture (PktmonCaptureDialog) ... 35
7.2 NETSH Trace (NetshTraceDialog) ... 35
8. Navigationsgruppe: Speedtests ... 37
8.1 Internet-Speedtest (InternetSpeedtestDialog) ... 37
8.2 Netzwerk-Speedtest (NetworkBandwidthDialog) ... 37
8.3 Disk-Speedtest (DiskCheckDialog) ... 37
9. Navigationsgruppe: E-Mail & Domain ... 38
9.1 E-Mail-Check (EmailCheckDialog) ... 38
9.2 E-Mail Header-Analyse (EmailHeaderAnalysisDialog) ... 38
9.3 E-Mail Domain-Analyse (EmailAnalysisDialog) ... 39
10. Navigationsgruppe: Inventar ... 40
10.1 Lokale Inventarisierung (LocalInventoryDialog) ... 40
10.2 Inventory Agent (InventoryAgentDialog) ... 41
11. Dienste & Hintergrundkomponenten ... 43
NetworkDiagnosticsService ... 43
NetworkOverviewService ... 43
DnsLookupService ... 43
MailDomainAnalyzer ... 43
MailSecurityAnalysisService ... 43
GraphMailTestService ... 44
DomainAnalysisService ... 44
FingerbankApiService ... 44
LocalDeviceIdentificationService ... 44
Etl2PcapngService ... 44
SettingsService ... 44
ExportService ... 44
SmtpTestService ... 44
SnmpService ... 45
12. NuGet-Pakete & Abhängigkeiten ... 46
13. Navigation & Dialog-Kommunikation ... 47
NavigateToSidebarRequested-Event ... 47
SetHostname-Methode ... 47
Lazy Loading & Caching ... 47
14. Deployment & Veröffentlichung ... 48
Build-Konfiguration ... 48

Publish-Profile 48
MSI-Installation (Setup) 48
Eingebetteter Agent 48
Eingebettete Ressourcen 48
Administrator-Rechte 49

© 2026 DATEXT

1. Projektübersicht

DATEXT Diagnostics ist ein umfassendes Windows-Diagnoseprogramm für IT-Administratoren und technisch versierte Anwender. Die Anwendung bietet in einer einzigen, lokal ausführbaren EXE-Datei ein breites Spektrum an Netzwerk-, System- und Performance-Werkzeugen – ohne Installation, ohne externe Abhängigkeiten zur Laufzeit.

Wichtigste Merkmale:

- Einzelne selbstständige EXE-Datei (Single-File-Publish, SelfContained); zusätzlich als MSI-Installation (WiX) mit Startmenü-Eintrag und Deinstallation
- Keine Installation erforderlich (portable EXE)
- Vollständig auf Windows 10/11 und Windows Server 2019+ ausgelegt
- Modernes WPF-UI mit einheitlichem Farbschema und Sidebar-Navigation
- Integrierte Exportfunktionen (PDF, CSV, JSON, TXT)
- Abschnittsweise Navigation über aufklappbare Gruppen in der Seitenleiste

© 2026 DATEXT

2. Technische Grundlagen

Plattform & Framework

Eigenschaft	Wert
Zielframework	.NET 8.0 (net8.0-windows)
UI-Technologie	WPF (Windows Presentation Foundation)
Ausgabe	WinExe (Desktop-Anwendung)
Nullable-Referenztypen	aktiviert
DPI-Awareness	PerMonitorV2 (via app.manifest)

Build & Veröffentlichung

Eigenschaft	Wert
Veröffentlichungstyp	Single-File
SelfContained	true
RuntimeIdentifier	win-x64
Komprimierung	aktiviert (EnableCompressionInSingleFile)
Native Libraries	eingebettet (IncludeNativeLibrariesForSelfExtract)

Assembly-Informationen

Eigenschaft	Wert
AssemblyName	DATEXT-Diagnostics
Version	0.99.12.11
Product	DATEXT Diagnostics
Company	DATEXT GmbH

© 2026 DATEXT

3. Architektur & Projektstruktur

Ordnerstruktur

```
DATEXT.Diagnostics/
├── App.xaml / App.xaml.cs            Anwendungseinstieg, globale Ressourcen
├── MainWindow.xaml / .cs             Hauptfenster, Sidebar-Navigation
├── Dialogs/
│   ├── NetworkDiagnostics/           Alle Netzwerk-Diagnose-Dialoge
│   ├── Mail/                         E-Mail-Check, E-Mail Header-Analyse, E-Mail Domain-Analyse
│   ├── SystemInfo/                   System-Informationsdialoge
│   ├── SystemHealth/                 Systemzustand-Dialog
│   ├── Performance/                  Geschwindigkeitstests
│   ├── ActiveDirectory/              Active-Directory-Analyse
│   ├── SqlAnalysis/                  MS-SQL-Analyse
│   ├── ServerServices/               IIS-Analyse, Hyper-V-Analyse, DNS-Server-Analyse, DHCP-Server-
Analyse, Terminalserver-Analyse
│   ├── Inventory/                    Lokale Inventarisierung, Inventory Agent, Inventar-Ansicht,
PDF-Berichte
│   ├── Troubleshooting/              Reparaturaktionen
│   ├── Updates/                      Winget-Paketverwaltung
│   └── (weitere Dialoge)
├── Controls/                         Wiederverwendbare Controls (Card, StatusRow, NavAssist, …)
├── Themes/                           Zentrales Style-Wörterbuch (Tokens, Typography, Controls,
Navigation)
├── Services/                         Hintergrundlogik & externe Dienste
├── Models/                           Datenmodelle (z. B. DnsLookupResult)
├── DATEXT.Shared/                    Gemeinsamer Code von Diagnostics und Agent
(Inventory/InventoryCollector.cs)
├── DATEXTAgent/                      Inventory Agent (Windows-Dienst, wird signiert in Diagnostics
eingebettet)
├── Setup/DATEXT.Diagnostics.Setup/   WiX-Projekt der MSI-Installation
└── Resources/                        Icons, Logos, MAC-Vendor-Datenbank, DATEXTAgent.exe
```

Die Oberflächen der E-Mail-Werkzeuge sind wie alle anderen `UserControl`s aufgebaut (früher war der E-Mail-Check ein eigenes `Window`). Große Dialoge sind in Partial-Dateien geteilt, z. B. `EmailCheckDialog.Tests/.Analysis/.Export/.Kim`, `InventoryAgentDialog.Deploy/.Scripts/.Agents/.Import` und `DnsLookupService.Checks/.Dnssec/.Servfail/.Services`.

Architekturprinzip

Jede Funktion ist als eigenständiger `UserControl` implementiert. Das `MainWindow` verwaltet die gesamte Navigation über eine Seitenleiste mit `RadioButton`-Gruppen. Dialoge werden per Lazy Loading (nur bei erster Aktivierung) instanziiert und danach gecacht. Kommunikation zwischen Dialogen erfolgt über typisierte `EventHandler`-Events (z.B. `NavigateToSidebarRequested`).

Navigation

Das `MainWindow` verwendet ein `switch`-Pattern über `nameof(rb...)` zur Auflösung des gewählten Navigationspunkts. Dialoge werden über `GetOrCreate<T>()` instanziiert – ein generischer Helfer, der ein Feld befüllt und optional eine Post-Aktion ausführt (z. B. Event-Verdrahtung).

```
private static T GetOrCreate<T>(ref T? field, Func<T> factory, Action<T>? post = null)
    where T : UIElement
{
    if (field == null) { field = factory(); post?.Invoke(field); }
    return field;
}
```

Update-Hinweis: Ein verfügbares Update zeigt ein Balken unter der Menüleiste („Version x ist verfügbar (installiert: y)“, „Release-Seite öffnen“, „Später“) und ein Chip „Update x“ rechts in der Menüleiste, der nach „Später“ sichtbar bleibt. Der frühere Eintrag in der Seitenleiste entfällt (der Stil `Nav.Notice` bleibt in `ThemesNavigation.xaml` ungenutzt erhalten). „Hilfe → Auf Updates prüfen“ und „Hilfe → Version x verfügbar“ bleiben.

Seitenleiste & zentrales Style-Wörterbuch

Seitenleiste (`MainWindow`): "Übersicht" plus sieben aufklappbare Gruppen – System, Netzwerk, Server & Dienste, Packet Capture, Speedtests, E-Mail & Domain, Inventar. Es ist immer höchstens eine Gruppe geöffnet; die Gruppe der aktiven Seite bleibt auch zugeklappt markiert (`NavAssist.IsActive`).

- **Suche** (Strg+F): durchsucht Seitennamen, Gruppennamen, Tooltips und `NavAssist.Keywords`; Gruppen mit Treffern klappen während der Suche auf, ohne Treffer erscheint "Keine Treffer".
- **Tastatur:** ↑/↓ bewegen nur den Fokus, Enter/Leertaste öffnet, →/← klappt Gruppen auf/zu, Pos1/Ende; Tab erreicht die Navigation als ein Element. Die Tasten wirken nur, wenn der Fokus in der Navigation liegt.
- **Kompakter Modus** (☰): 48 px, nur Icons; Gruppen öffnen ein Flyout-Menü. Gespeichert in `%AppData%\DATEXT-Diagnostics\sidebar-compact.flag`.
- **Status-Punkt** (`NavAssist.Badge`): z. B. an "Übersicht" bei Hinweisen/Problemen der System-Übersicht.
- **Deep-Links** aus Dialogen (`NavigateToSidebarRequested`) klappen die Zielgruppe über `ExpandGroup()` auf.

Zentrales Style-Wörterbuch (`Themes/`, eingebunden in `App.xaml`): `Tokens.xaml` (Farben mit Status-Semantik, Abstände, Radien, Icon-Schrift `Segoe Fluent Icons, Segoe MDL2 Assets`), `Typography.xaml` (`Text.*`), `Controls.xaml` (`Button.*`, `Input.TextBox`, `Input.DatePicker` mit Kalender-Button außerhalb des Feldes, `Input.ComboBox`, `Tab.Item`, `Badge*`, `Chip*`, `ProgressBar.Usage`, `ctl:Card`, `ctl:StatusRow` – mit `SingleLineDetail="True"` einzeilig gekürzt, vollen Text als ToolTip) und `Navigation.xaml` (`Nav.*`). Code-Bausteine in `Controls/` (`Card`, `StatusRow`, `ButtonAssist`, `NavAssist`, `Theme`). Neue Dialoge verwenden ausschließlich diese Keys – keine lokalen Style-Kopien, keine hartcodierten Farben/Schriftgrößen. Referenz: `SystemOverviewDialog`.

4. Navigationsgruppe: System

4.1 Übersicht (SystemOverviewDialog)

Klasse: `SystemOverviewDialog` (Dialogs/SystemInfo/) **Technologie:** WMI (`System.Management`), `NetworkInterface`, HTTP

Der System-Übersichtsdialog lädt parallel alle wesentlichen Systeminformationen:

- **Systemdaten:** CPU-Modell, Arbeitsspeicher, Betriebssystem-Version, Hostname, Domäne (via WMI `Win32_ComputerSystem`, `Win32_Processor`, `Win32_OperatingSystem`)
- **Netzwerkdaten:** Primärer Adapter, IP-Adresse, Gateway, DNS-Server (via `NetworkOverviewService`)
- **Internet-Status:** Externe IP-Adresse, Geolokation (via HTTP-Abfrage)
- **Zeitinformationen:** Aktuelle Systemzeit, Zeitzone, NTP-Synchronisation

Export ist als PDF, CSV und JSON möglich. Ein Button ermöglicht die direkte Navigation zum Internet-Speedtest (über das `NavigateToSpeedtestRequested`-Event).

Oberfläche: Referenz-Dialog des zentralen Style-Wörterbuchs (siehe Kapitel 3, "Seitenleiste & zentrales Style-Wörterbuch"). Status-Chips oben mit Gesamtstatus ("Keine Auffälligkeiten" / "N Hinweise"); Informations-Chips neutral, farbig nur bei Auffälligkeiten. Netzwerkzone, Antivirus, Firewall (je Profil Domäne/Privat/Öffentlich) und Windows Update als Statuszeilen (`StatusRow`). IP-/DNS-Zeilen mit Kopieren-Symbol und "⋯"-Menü (Ping, Traceroute, IP & DNS Analyse, Port-Scan). Unter 980 px Breite einspaltig. Der Gesamtstatus wird über das Event `SummaryStateChanged` an die Navigation gemeldet (Status-Punkt an "Übersicht").

4.2 System-Health (SystemHealthDialog)

Klasse: `SystemHealthDialog` (Dialogs/SystemHealth/) **Technologie:** Windows-API, Registry, WMI und COM; PowerShell nur noch für zwei Sonderfälle (Hyper-V-Switch-Bindung auf Hosts mit vEthernet-Adapter, Server-Rollen auf Windows Server)

Prüft den Gesundheitszustand des Systems in acht Bereichen: Hardware, Betriebssystem, Speicher, Sicherheit, Netzwerk, Performance, Dienste und Ereignisse. Die Prüfungen sind nach dem Malwarebytes-Vorfall weitgehend nativ umgesetzt (sprachunabhängig, schneller, kleinerer Heuristik-Fußabdruck; vollständiger Scan getestet: 72 s).

- **Native Quellen (Auswahl):** Defender (`MSFT_MpComputerStatus`), Firewall-Profile (Registry inklusive Gruppenrichtlinie), SMB1 und PowerShell 2.0 (`Win32_OptionalFeature`), RDP/NLA (inklusive Richtlinie), VBS/Credential Guard (`Win32_DeviceGuard`), NIC-Statistik, Link und Offload (`NetworkInterface` plus Treibereinstellungen), Routing (`GetIpForwardTable`), ARP (Nachbartabelle), Netzwerkzone (NetworkListManager), Firewall-Regeln (`HNetCfg.FwPolicy2`), Energieplan (`powrprof`), Commit (`GetPerformanceInfo`), DPC/ISR (PDH), Stabilitätsindex und Auslagerungsdatei (WMI), geplante Aufgaben (Taskplaner-COM), .NET-Laufzeiten (Installationsordner).
- **Neue Prüfungen:** Defender-Signaturen älter als 7 Tage (Warnung), Manipulationsschutz aus (Hinweis), letztes Update älter als 45 Tage (Warnung), Neustart ausstehend auch nach Komponentenwartung (CBS), UAC-Stufe im Klartext („ohne Rückfrage hochstufen“ als Warnung), RAM-Typ einschließlich LPDDR und eingestelltem Takt, `MEMORY.DMP` neben den Minidumps.
- **Bereichs-Kacheln** unter der Werkzeugleiste mit Anzahl Probleme/Warnungen; ein Klick öffnet den Tab.
- **Gesamtbewertung** als Punktzahl („65 / 100“: 100 − 15 je Problem − 4 je Warnung); bei Abbruch als unvollständig gekennzeichnet.

- **Problemliste:** Fehler vor Warnungen, ein Klick öffnet den zugehörigen Tab. Die Karte „Nicht geprüft“ nennt Bereiche, die mangels Rechten nicht ausgewertet werden konnten; sie fließen nicht in die Bewertung ein.
- **Export** ohne Emojis („[OK]“, „[!]“, „[X]“, „[i]“) und mit der Liste der nicht geprüften Bereiche. Status-Icons sind Segoe Fluent in Theme-Farben (zentraler Converter).

Ergebnisse werden als `DiagItem`- und `ProblemItem`-Objekte in `ObservableCollections` gebunden.

4.3 UEFI-Boot-Zertifikat (UefiBootCertDialog)

Klasse: `UefiBootCertDialog` (Dialogs/SystemInfo/) **Technologie:** Windows-API ohne PowerShell: `GetFirmwareEnvironmentVariableEx` (mit `SeSystemEnvironmentPrivilege`), TBS-API (`Tbsi_GetDeviceInfo`), Shell-Eigenschaft `System.Volume.BitLockerProtection`, Registry

Liest und bewertet die UEFI-Secure-Boot-Zertifikate und den Stand der Zertifikatsumstellung 2011 → 2023.

- **Quellen:** Zertifikate in db, KEK und PK direkt aus der Firmware. Der Boot-Manager wird über den Volume-GUID-Pfad der EFI-Partition geprüft (`X509Certificate.CreateFromSignedFile`); als Administrator läuft die Prüfung automatisch. TPM über die TBS-API und BitLocker über die Shell-Eigenschaft (beides ohne Administratorrechte).
- **Kennzahl-Kacheln:** Secure Boot, Umstellung 2023, KEK 2023, Windows UEFI CA 2023, Boot-Manager, Tage bis zum Ablauf von PCA 2011.
- **Karte „Zertifikatsumstellung 2011 → 2023“:** je Rolle (KEK, Windows-Boot, Drittanbieter, Option-ROMs, Boot-Manager), ob der 2023-Nachfolger vorhanden ist.
- **Karte „Windows-Servicing“:** Umstellungsstatus, Rollout-Steuerung (AvailableUpdates/-Policy, HighConfidenceOptOut, MicrosoftUpdateManagedOptIn, ConfidenceLevel), Task mit letzter Ausführung, Fehlercodes (inklusive `KEKLastUpdateError`/`-Reason`) und Ereignisliste mit Bedeutung, Anzahl und Zeitpunkt.
- **Bewertung nach Rolle:** Ein ersetztes 2011-Zertifikat erscheint als „Ersetzt“; ein abgelaufener PK oder ein fremdes abgelaufenes Zertifikat als „Ohne Wirkung“, denn die Firmware prüft Ablaufdaten nicht. Status der übrigen Zertifikate: OK, WarningSoon (bald ablaufend), Expired, LegacyExpired.
- **Export** als PDF (Standard), CSV (mit Rollentabelle, BOM), JSON (`System.Text.Json`, mit Rollen, Ereignissen, Task und Boot-Manager) und Zwischenablage; der Speicherort ist per Dialog wählbar (`ExportFile`).

4.4 Autorun-Analyse (AutorunAnalysisDialog)

Klasse: `AutorunAnalysisDialog` (Dialogs/SystemInfo/) **Technologie:** Registry, TaskScheduler-NuGet, `System.ServiceProcess`, WMI

Listet Autostart-Orte des Systems und bewertet sie (Signatur, fehlende Datei, Risiko):

- **Registry:** Run und RunOnce (HKCU, HKLM, 32 Bit), `Policies\Explorer\Run`, Debugger-Umleitungen (Image File Execution Options), `AppInit_DLLs` (mit `LoadAppInit_DLLs`), `BootExecute` (ohne den Windows-Standard `autocheck autochk *`), Active Setup. Winlogon zeigt nur `Shell`, `Userinit`, `Taskman` und `AppSetup` und meldet Abweichungen vom Windows-Standard.
- **WMI-Ereignisabonnements** mit Befehls- oder Skript-Consumer (dateilose Persistenz; nur mit Administratorrechten lesbar).
- **Geplante Aufgaben:** alle Tasks mit allen Aktionen, ein Eintrag je Aktion. COM-Handler werden bis zur DLL aufgelöst. Auslöser und Zustand auf Deutsch, dazu der Ordner des Tasks.
- **Dienste:** unterscheidet „Automatisch (verzögert)“; bei svchost-Diensten wird die eigentliche Dienst-DLL geprüft.
- **Autostart-Ordner.**

- **Zustand wie im Task-Manager:** `StartupApproved` wird gelesen. Run-Einträge und Autostart-Ordner lassen sich per Rechtsklick deaktivieren und wieder aktivieren (umkehrbar, Byte-Format identisch mit dem Task-Manager). Für HKLM-Einträge und den gemeinsamen Autostart-Ordner wird der Neustart als Administrator angeboten. Früher per Umbenennung in `.disabled` deaktivierte Verknüpfungen werden erkannt und lassen sich wiederherstellen.
- **Oberfläche:** Kennzahl-Kacheln (Einträge, Gefahr, Warnung, nicht signiert, Datei fehlt, deaktiviert; Klick filtert), Suchfeld und Filter „Microsoft-Einträge ausblenden“ (Standard an, wie bei Sysinternals Autoruns), Spalte „Hinweis“ mit dem Grund der Einstufung, Tab-Zähler für Gesamt und Auffällige. Der Scan lässt sich abbrechen; ohne Administratorrechte nennt ein Hinweis, was fehlt.
- **Export:** CSV (mit BOM), PDF und Zwischenablage mit der Liste nicht geprüfter Bereiche.

4.5 Ereignislog-Viewer (EventLogDialog)

Klasse: `EventLogDialog` (Dialogs/SystemInfo/) **Technologie:** `System.Diagnostics.EventLog`

Liest Windows-Ereignisprotokolle (System, Anwendung, Sicherheit) und stellt sie in einer filterbaren Tabelle dar. Ereignisse werden als `EventLogEntry2`-Objekte in einer Liste gehalten. Filter nach Schweregrad (Fehler, Warnung, Information) und Zeitraum sind verfügbar. Die Abfrage läuft direkt über `EventLogReader` mit XPath (Schweregrad, Anbieter, IDs und Zeitraum werden beim Lesen gefiltert).

Schnellfilter (`EventLogDialog.Scenarios`) wählen Protokoll, Anbieter und Ereignis-IDs für typische Fragen vor. Die Gruppen sind in der Auswahlliste einklappbar (Gruppenzeile mit Stil `Input.ComboBoxHeader.Collapsible`; `SetScenarioGroupOpen` schaltet die Sichtbarkeit der Einträge um, beim Öffnen wird die Gruppe des gewählten Filters aufgeklappt). Neuere Filter: „Neue Dienste und geplante Aufgaben“ (7045, 4697, 4698, 4702, 106; mehrere Protokolle), „PowerShell-Aktivität“ (4103/4104), „Schutzfunktionen verändert“ (Defender 5001/5004/5007/5010/5012, Firewall 2003–2006), „Hardwarefehler (WHEA)“ und „Datenträger-Zurücksetzungen“ (storahci/stornvme 129), „Prozessor gedrosselt (Firmware)“ (Kernel-Processor-Power 37), „Sicherung und Schattenkopien“ (Backup, VSS, volsnap), „Blockierte Programme (AppLocker, Code Integrity)“, „WLAN-Verbindungen“ (8001–8003), „Netzwerkprofil und DHCP-Client“, „TLS und Zertifikate (Schannel)“ (36874/36882/36887/36888), „SMB-Client-Verbindungen“, „Windows-Fehlerberichte (WER)“ (1001) und „Store-Apps (AppX)“. Weitere rollenabhängige Gruppen am Ende der Liste: **DHCP-Server** (Dienst `DHCPServer`; Kanäle `DHCP Server Events/Admin` und `/Operational`), **NPS (RADIUS)** (Dienst `IAS`; Sicherheitsereignisse 6272–6278) und **Dateiserver (SMB)** (nur auf Serverbetriebssystemen; Kanäle `SMBServer/Security`, `/Connectivity`, `/Operational`). Gruppen in dieser Reihenfolge: Anwendungen, System & Stabilität, Sicherheit & Konten, Netzwerk, **Active Directory**. Die Gruppe Active Directory ist für Domänencontroller gedacht: Replikation (AD DS), SYSVOL / DFS-Replikation, LDAP-Signing und unsichere Binds (Ereignisse 2886–2889), AD-integriertes DNS, Netlogon und DNS-Registrierung, Kerberos und NTLM am DC, Kerberos-Fehler (KDC / Client) sowie Änderungen an AD-Objekten (5136–5141). Auf Servern mit der jeweiligen Rolle kommen vier weitere Gruppen hinzu, die auf anderen Rechnern ausgeblendet sind (Reihenfolge: Hyper-V, IIS, Cluster, SQL Server): **Hyper-V** (VM-Betrieb, Speicher und virtuelle Festplatten, virtueller Switch, Hypervisor und Konfiguration, Hochverfügbarkeit und Live-Migration – fassen die Hyper-V-Protokolle je Thema zusammen und zeigen alle Ereignisse, eingegrenzt wird über die Schweregrad-Schaltflächen), **IIS** (erscheint mit dem Dienst `W3SVC`: Anwendungspools und Arbeitsprozesse über den Anbieter Microsoft-Windows-WAS, Websites und Dienst über W3SVC und W3SVC-WP, ASP.NET-Fehler, HTTP.sys und Zertifikate über Microsoft-Windows-HttpEvent und -HttpService, ASP.NET über die Anbieter „ASP.NET 4.0.30319.0“ und „ASP.NET 2.0.50727.0“ sowie der Kanal Microsoft-IIS-Configuration/Operational; die Filter zeigen alle Ereignisse des jeweiligen Anbieters, im Detailbereich erklärt ein Hinweis die Ereignisse 5002, 5011, 5057 und 5059 des Windows Process Activation Service), **Cluster** (Knoten und Quorum, Ressourcen und Rollen, Clusternetzwerk, Cluster Shared Volumes, Cluster-Diagnose) und **SQL Server** (Anmeldefehler, Datenbank beschädigt oder nicht verfügbar, Speicher, Sicherung, Always On, SQL Server Agent, alle Ereignisse; Standardinstanz und benannte Instanzen). Für die wichtigsten Cluster- und SQL-Ereignisse zeigt der Detailbereich einen Hinweis. Die Protokolle „Directory Service“, „DFS Replication“ und „DNS Server“ stehen fest in der Protokollauswahl und gibt es nur auf DCs bzw. DNS-Servern; auf anderen Rechnern meldet der

Dialog, dass das Protokoll fehlt. Die Filter der AD-Protokolle verzichten auf den Anbieternamen (er wechselt je Windows-Version), weil die IDs im eigenen Protokoll eindeutig genug sind.

Mehrere Protokolle: Ein Protokollschlüssel der Form `multi:System|Application|Security` steht für mehrere Protokolle (`EventLogDialog.IsMulti`, `SplitLogs`). `ReadEventsMulti` liest jedes Protokoll mit derselben XPath-Abfrage, führt die Treffer nach Zeit zusammen und begrenzt auf die gewählte Anzahl; nicht lesbare oder fehlende Protokolle werden übersprungen und in der Statuszeile genannt. EVTX-Export ist dafür gesperrt. Die Auswahl „Weitere …“ erlaubt eine Mehrfachauswahl.

Zeitvorgaben: `count:N` (letzte N Treffer), `lasth:N` (letzte N Stunden), `last:N` (letzte N Tage), `today`, `yesterday` und `timerange` (von/bis mit Datum und Uhrzeit). Zeitabfragen sind auf 50.000 Treffer begrenzt (`TimeRangeCap`).

Ansichten (`EventLogDialog.Features.cs`): Die Liste wird in zwei Stufen gefiltert: `_viewBase` (Suche, „Markierte“) und `_filtered` (zusätzlich das Zeitfenster aus der Zeitleiste). Exporte arbeiten auf `_filtered`.

- *Zusammenfassung:* `SummaryRow` je Protokoll, Quelle und ID, mit schwerstem Schweregrad, Anzahl, erstem/letztem Auftreten und Beispielmeldung.
- *Zeitleiste:* Canvas mit gestapelten Balken (Fehler, Warnung, Information). Die Balkenbreite ist die kleinste „runde“ Zeitspanne (1 Minute bis 30 Tage), bei der etwa ein Balken je 10 px entsteht. Klick oder Ziehen setzt `_timeWindow`; der Knopf im Chip lädt den Zeitraum über `timerange` neu aus dem Protokoll.
- *Live-Modus:* je Protokoll ein `EventLogWatcher` mit der XPath-Abfrage ohne Zeitraum. Das Ereignis wird sofort in einen `EventLogEntry2` umgewandelt und in eine Warteschlange gelegt; ein `DispatcherTimer` (700 ms) fügt die Einträge vorn ein (höchstens 50.000), solange nicht pausiert ist. Beim Schließen des Dialogs und beim Wechsel in die Kontext-Ansicht werden die Watcher beendet.
- *Ereignisdaten:* `ReadDetails` zerlegt EventData/UserData des XML in Name/Wert-Paare (`ParseFields`); die Tabelle sitzt rechts neben dem Detailtext.
- *Hinweise:* `EventLogKnowledge` (79 Einträge zu 123 IDs) ordnet Quelle (Teilstring) und ID einem Titel, einer Bedeutung und einer Empfehlung zu.
- *Lesezeichen und Notizen:* `EventLogMarks` speichert sie in `%APPDATA%\DATEXT\DATEXT.Diagnostics\eventlog-marks.json` (Schlüssel: Protokoll, Datensatznummer, Zeitstempel). `EventLogEntry2` meldet Änderungen per `INotifyPropertyChanged` an die Spalte „★“.

Standardumfang und Kacheln: Voreingestellt sind die letzten 6 Stunden (`DefaultRangeTag = "lasth:6"`); zusätzlich wählbar sind u. a. „Letzte Stunde“, 24 Stunden, 7 und 30 Tage, „Heute“, „Gestern“, „Letzte 50 … 20.000 Treffer“, „Anzahl eingeben …“ (ohne gültige Eingabe 1000) und „Von / bis …“. Kennzahl-Kacheln statt einer linken Kartenspalte: Kritisch, Fehler, Warnungen, Informationen und „Überwachung gescheitert“ sind zugleich der Schweregrad-Filter (Klick blendet den Schweregrad aus und lädt neu); dazu „Häufigste Quelle“ und „Zeitraum der Treffer“. Die Zeile „Häufigste Fehler/Warnungen“ (Quelle, ID, Anzahl) lädt per Klick nur diese; die Einschränkung erscheint als entfernbares Merkmal, ebenso über Rechtsklick „Nur diese Quelle (und ID) laden“. „Filter zurücksetzen“ setzt Schweregrade, Schnellfilter, Einschränkung, Zeitfenster und Suche zurück, „Liste leeren“ nur die Ergebnisse.

Protokolle: Direkt wählbar sind System, Anwendung, Sicherheit, Setup, Windows PowerShell, PowerShell (Operational), Defender, Aufgabenplanung, RDP-Sitzungen, RDP-Verbindungen und Druckdienst; „Weitere …“ durchsucht alle Ereignisprotokolle des Computers. Ein deaktiviertes Protokoll kann der Dialog aktivieren („Protokoll aktivieren“, ohne Administratorrechte mit Neustart als Administrator).

Detail und Kontext: Das Detail zeigt Nachricht, Kategorie, Schlüsselwörter, Benutzer (aus der SID aufgelöst), Computer, Prozess-ID und Datensatznummer (für den gewählten Eintrag nachgeladen) und lässt sich auf „XML“ (Rohdaten) umschalten. Das Kontextmenü bietet „Kontext anzeigen (±5 Min., alle Hauptprotokolle)“ (Kontext-Ansicht, in der der Live-Modus nicht verfügbar ist, „Zurück zur vorherigen Ansicht“), Lesezeichen und Notiz, „In den Ergebnissen nach diesem Wert suchen“, „Im Web suchen“ (Google, Bing, Microsoft

Learn, Microsoft Q&A) und für das Sicherheitsprotokoll die Security-Enzyklopädie. eventid.net entfällt (Domain nicht mehr erreichbar).

Export: CSV-Tabelle (alle Felder, vollständige Nachricht, BOM), EVTX (echte Ereignisprotokoll-Datei inklusive Meldungstexten für die Windows-Ereignisanzeige; bei begrenzter Anzahl gilt der Zeitraum der geladenen Treffer, für das Sicherheitsprotokoll nur mit Administratorrechten; bei mehreren Protokollen gesperrt), TXT-Datei und Zwischenablage (vollständiger Text mit Abfragekopf, höchstens 2000 Einträge zum Kopieren) sowie PDF-Bericht (höchstens 1000 Einträge mit Hinweis auf den Rest). Der **Support-Ticket-Auszug** ist eine kompakte, wahlweise anonymisierte Zusammenfassung (`Anonymize`: Computer- und Domänenname, Benutzername, Konten-SIDs, IPv4- und IPv6-Adressen). Exporte nennen ein gewähltes Zeitfenster und arbeiten auf der gefilterten Liste.

4.6 Installierte Software (SystemSoftwareDialog)

Klasse: `SystemSoftwareDialog` (Dialogs/SystemInfo/) **Technologie:** Registry (Uninstall-Schlüssel), `msiexec`

Zeigt die installierten Programme wie „Programme und Features“. Gelesen werden die Uninstall-Schlüssel von HKLM (64 und 32 Bit) und HKCU (Benutzerinstallationen); gleichnamige Einträge in 64 und 32 Bit werden nicht zusammengelegt. Das Lesen läuft im Hintergrund (Fortschrittsanzeige) und ist per `LoadIfNeeded()` verzögert geladen; die Softwareliste kann vom System-Übersichtsdialog für den PDF-Export genutzt werden.

- **Standardansicht:** Systemkomponenten (`SystemComponent = 1`) sowie Teilprodukte und Updates (`ParentKeyName`, `ReleaseType`) sind ausgeblendet; der Schalter „Systemkomponenten und Updates anzeigen“ blendet sie ein. Auf einem Testsystem waren 117 statt 416 Einträge sichtbar.
- **Kennzahl-Kacheln:** Programme (mit Anzahl Hersteller), letzte 30 Tage, nur dieser Benutzer, 32 Bit, verwaist, Gesamtgröße; ein Klick filtert.
- **Spalten:** Name mit Programmsymbol (aus `DisplayIcon`), Herausgeber, Version (numerisch sortiert), Datum, Größe (MB/GB), Bereich (Alle Benutzer 64/32 Bit, nur dieser Benutzer) und Hinweis. Fehlt `InstallDate`, steht der Zeitpunkt der letzten Änderung des Registry-Eintrags (grau-kursiv mit Tooltip). Verwaiste Einträge (Deinstallationsprogramm fehlt) sind markiert.
- **Detailbereich:** Installationsordner, Deinstallationsbefehl, MSI-Produktcode, Registry-Schlüssel, Website.
- **Aktionen (Kontextmenü):** Deinstallieren (startet nach Bestätigung das Deinstallationsprogramm des Eintrags, bei MSI `msiexec /x {Produktcode}`; eine stille Deinstallation wird bewusst nicht angeboten), Ändern/Reparieren (`ModifyPath` bzw. `msiexec /i`), Installationsordner, In Registry öffnen, Hersteller-Seite, Websuche, Name bzw. Deinstallationsbefehl kopieren. `NoRemove`/`NoModify` werden beachtet; nach Abschluss wird die Liste neu gelesen.
- **Filter:** Herstellerfilter mit Platzhalter, „Filter zurücksetzen“, Suche (auch im Installationsordner, 200 ms Verzögerung).
- **Export** der aktuellen Ansicht (Kopf nennt die aktiven Filter) als PDF, CSV (alle Felder, BOM) und JSON (ISO-Datum); die System-Übersicht übernimmt nur Programme ohne Systemkomponenten.

4.7 Winget-Pakete (WingetDialog)

Klasse: `WingetDialog` (Dialogs/Updates/), Parser in `WingetDialog.Parsing.cs` **Technologie:** `winget.exe` CLI (alle Aufrufe mit `--disable-interactivity --accept-source-agreements`), Tabellen-Parser, GitHub-API für die Selbstaktualisierung von winget

Verwaltet Software über den Windows Package Manager in drei Tabs: **Updates**, **Alle Pakete** und **Katalog & Suche**. Gestartet wird mit `winget --version`; fehlt winget, erscheint ein Hinweis.

- **Laden:** `winget upgrade --include-unknown --include-pinned`, `winget list` und `winget pin list` laufen parallel. Der Parser (`upgrade`, `list`, `search`, `pin list`) liest die Spalten aus der Kopfzeile

(deutsch und englisch) und schneidet nach Anzeigebreite (auch doppelt breite Zeichen); mehrere Tabellen und Fortschrittsreste werden erkannt. Details (`winget show`, 2–3 s) werden je Paket zwischengespeichert; deutsche und englische Feldnamen werden ausgewertet.

- **Kennzahl-Kacheln:** Updates (davon mit Pin), Installiert, Über winget, Ohne Quelle (ARP/MSIX), Mit Pin, Zuletzt geprüft (mit winget-Version). Ein Klick wechselt in den Tab bzw. Filter.
- **Updates-Tab:** Auswahl per Kontrollkästchen. „Ausgewählte aktualisieren“, „Alle aktualisieren“ (Pin-geschützte Pakete werden übersprungen und in der Bestätigung genannt) und im Dropdown „Auch PIN-geschützte Pakete einbeziehen …“. Pakete mit blockierendem Pin (`Blocking`) werden übersprungen („zuerst den Pin entfernen“).
- **Alle Pakete:** Spalten Status (farbiger Punkt), Name, ID, Version, Verfügbar, Quelle, Pin-Art (Pinning/Blocking/Gating); Filter „Nur mit Update / Über winget verwaltbar / Ohne Quelle / Mit Pin“ und Textfilter. Detailbereich mit Herausgeber, Beschreibung, Startseite und Lizenz.
- **Kontextmenü:** Update prüfen/installieren, Deinstallieren, PIN setzen/entfernen, Installationsordner öffnen, ID und Pfad kopieren.
- **Katalog & Suche:** Katalog mit 45 vorbereiteten Anwendungen in acht Gruppen (Browser, E-Mail & Kommunikation, Sicherheit & VPN, System & Diagnose, Office & Produktivität, Medien & Grafik, Entwicklung, Cloud & Backup); bereits installierte Pakete sind markiert. Freie Suche (`winget search`) mit getrennter Übereinstimmung und Hinweis „installiert“; Mehrfachauswahl und „Ausgewählte installieren“.
- **Ergebnis je Paket:** Aktualisieren, Installieren und Deinstallieren laufen über einen gemeinsamen Ablauf (`RunBatchAsync`; `upgrade/install --id … --exact --silent --accept-package-agreements`, `uninstall --id … --exact --silent`). Jeder winget-Exit-Code wird ausgewertet (`DescribeExitCode`): erfolgreich, nicht anwendbar (bereits aktuell/installiert), Neustart erforderlich oder fehlgeschlagen mit Klartext (u. a. Anwendung läuft, andere Installation läuft, Datei in Benutzung, Abhängigkeit fehlt, Datenträger voll, zu wenig Arbeitsspeicher, Richtlinie, Prüfsumme, UAC abgelehnt). Am Ende steht eine Zusammenfassung („19 erfolgreich, 1 mit Neustartbedarf, 1 fehlgeschlagen“) mit der Liste der Ausnahmen. Das Neustart-Banner („Neustart erforderlich“, „Jetzt neu starten“) richtet sich nach dem Exit-Code.
- **Abbruch:** „Stopp“ beendet den gesamten Prozessbaum (winget und Installer); verbleibende Pakete gelten als abgebrochen.
- **Administratorrechte:** Hinweis „Einige Updates benötigen Admin-Rechte“; „Als Administrator neu starten“ gibt die Einzelinstanz-Sperre frei.
- **winget-Selbstaktualisierung:** Der Dialog fragt die neueste Version über die GitHub-API (`microsoft/winget-cli`) ab. Ist eine neuere verfügbar, zeigt ein Banner „Update für winget (App Installer) verfügbar“ mit „Im Microsoft Store öffnen“ (`ms-windows-store://pdp/?productid=9NBLGGH4NNS1`) oder „Herunterladen & installieren“ (`.msixbundle` aus dem Release, Installation mit `Add-AppxPackage`).
- **Protokoll:** Ausgabe „WINGET-AUSGABE“ ein-/ausblendbar, mit Kopieren und Leeren; Zeilen mit „[OK]/[!]/[X]/[i]“.
- **Export:** CSV aller Pakete (Name, ID, installiert, verfügbar, Update, Quelle, Pin), TXT und PDF mit Computer, Abfragezeit, Kennzahlen, Quelle und Pin-Art.

4.8 System-Verwaltung (SystemServicesDialog)

Klasse: `SystemServicesDialog` (Dialogs/SystemInfo/) **Technologie:** Prozessstart mit ShellExecute, datengetriebene Werkzeugliste im Code

Schnellzugriff auf 45 Windows-Verwaltungswerkzeuge in sechs Gruppen:

- **Verwaltung:** u. a. Dienste, Gerätemanager, Computerverwaltung, Registrierungseditor, Aufgabenplanung, Lokale Benutzer und Gruppen, Gruppenrichtlinie, Sicherheitsrichtlinie, Zertifikate (Computer und Benutzer), Freigaben, Druckverwaltung.

- **Überwachung & Diagnose:** Ressourcenmonitor, Leistungsüberwachung, Zuverlässigkeitsverlauf, Systeminformationen.
- **Sicherheit & Updates:** Windows-Sicherheit, Firewall (erweitert), Windows Update.
- **Netzwerk:** Netzwerk-Einstellungen, Internetoptionen, Remote-Einstellungen.
- **System & Einstellungen:** Umgebungsvariablen, Programme und Features, Windows-Features, Systemwiederherstellung (`rstrui.exe`) und Windows-Wiederherstellung (`ms-settings:recovery`), Datenträgerbereinigung.
- **Benutzer & Konten:** Konten (Einstellungen).

Beim Öffnen wird die Verfügbarkeit geprüft: Fehlt eine Datei oder ist das Tool in Windows Home nicht enthalten (Gruppenrichtlinie, Sicherheitsrichtlinie, Lokale Benutzer, Druckverwaltung, BitLocker), erscheint die Kachel ausgegraut mit Begründung. Ein Schild-Symbol markiert Werkzeuge, die Administratorrechte anfordern. Rechtsklick: „Als Administrator starten“ und „Befehl kopieren“. Ein Suchfeld durchsucht Name, Befehl und Beschreibung; die Eingabetaste öffnet den ersten Treffer. Bricht der Benutzer die Abfrage nach Administratorrechten ab, meldet die Statuszeile das. Die Outlook-Profile (Mail) werden über die CPL im Office-Ordner geöffnet (Click-to-Run).

4.9 Troubleshooting (TroubleshootingDialog)

Klasse: `TroubleshootingDialog` (Dialogs/Troubleshooting/) **Technologie:** DISM (`dism.exe`), SFC (`sfc.exe`), `netsh`, `shutdown`, `w32tm`, Dienststeuerung, Registry

Sammlung einzeln auslösbarer Reparaturaktionen – jede Aktion startet per eigenem Button (keine Sammelausführung). Benötigt Administratorrechte: Werkzeuge, die diese brauchen, sind ohne sie gesperrt und markiert; „Als Administrator neu starten“ gibt die Einzelinstanz-Sperre frei. Das Protokoll ist immer sichtbar (Kopieren, Speichern, PDF „Reparaturprotokoll“); Status und „Abbrechen“ stehen im Kopfbereich, die Gruppen sind auf- und zuklappbar.

- **Windows Update:** Update-Ordner, BITS-Warteschlange und DLL-Registrierung zurücksetzen (gebündelt), Update-Richtlinien zurücksetzen, `gpupdate`, Bereinigung des Komponentenspeichers – optional mit `/ResetBase` (mit Bestätigung, da nicht umkehrbar), alte Update-Sicherungen löschen (`SoftwareDistribution.bak_*`, `Catroot2.bak_*`).
  - Update-Richtlinien: Die Bestätigung listet die betroffenen Werte (auch WSUS-Server) und weist bei Domänenmitgliedern darauf hin, dass `gpupdate` die Werte der Domänen-GPOs zurückbringt. Vorher entsteht eine `.reg`-Sicherung unter `%ProgramData%\DATEXT-Diagnostics\Backups`; schlägt sie fehl, wird nichts gelöscht.
  - Komponenten neu registrieren: 13 Update- und Krypto-DLLs; fehlende DLLs und DLLs ohne Registrierungsfunktion werden übersprungen.
  - Der Update-Ordner-Reset weist auf vorhandene alte Sicherungsordner und deren Größe hin; ein Abbruch der DISM-Bereinigung wird als unvollständig erklärt.
- **Netzwerk-Reset:** `netsh int ip reset`, `netsh winsock reset` und `netsh winhttp reset proxy` (zeigt die aktuelle Einstellung) als getrennte Aktionen. Die Bestätigung des TCP/IP-Resets zeigt alle Adapter mit statischer IPv4-Adresse, die dabei verloren geht; bei installierter Hyper-V-Rolle (Dienst `vmms`) ist eine gesonderte Bestätigung nötig (vSwitch-Bindungen, NIC-Teaming, VLAN).
- **Weitere Werkzeuge:** Druckwarteschlange leeren, Uhrzeit synchronisieren (`w32tm /resync`), Temp-Bereinigung (eine Bestätigung mit Anzahl und Größe; `%TEMP%\.net` bleibt ausgenommen), Windows-Problembehandlungen.
- **Neustart:** „In 15 Sekunden“ (`shutdown /r /t 15`; Windows schließt alle Programme ohne Rückfrage, was die Bestätigung nennt) und „Jetzt neu starten“ (`/r /t 0`); der Exit-Code von `shutdown` wird geprüft. Die Empfehlung zum Neustart erscheint nur nach TCP/IP- bzw. Winsock-Reset.

- **Systemdatei- & Image-Integrität** (eingebettet, auf-/zuklappbar; früher eigener Navigationspunkt „Windows Komponenten Check“): Reihenfolge DISM zuerst, SFC zuletzt – `/CheckHealth` → `/ScanHealth` → ggf. `/RestoreHealth` (Live-Auswertung der CBS.log) → `sfc /scannow`.

Zuverlässigkeitsverlauf, Leistungsüberwachung, Systeminformationen und Internetoptionen sind in die System-Verwaltung verschoben. Alle Operationen laufen asynchron mit Abbruchmöglichkeit; die Ausgabe der Werkzeuge landet im Protokoll, das Ergebnis steht in der Schrittkarte.

© 2026 DATEXT

5. Navigationsgruppe: Netzwerk

5.1 Geräte-Scan (DeviceScanDialog)

Klasse: `DeviceScanDialog` (Dialogs/NetworkDiagnostics/) **Technologie:** ICMP-Ping, Windows-Nachbartabelle (`GetIpNetTable2`), ARP (`SendARP`), mDNS/SSDP/WS-Discovery (UDP-Multicast), TCP-Banner, SMB2/NTLMSSP, SNMP v2c/v3, UniFi-API, FRITZ!Box-TR-064, Fingerbank-API

Der umfangreichste Netzwerkdialog. Er scannt ein oder mehrere IP-Bereiche auf aktive Geräte und identifiziert sie.

Scan-Bereich: „Aktuelles Subnetz scannen“ (Adapterauswahl über `ctl:AdapterList`), CIDR-Notation (z. B. `192.168.1.0/24`) oder von/bis; mit „Bereich hinzufügen“ lassen sich mehrere Bereiche kombinieren.

Ablauf eines Scans (`BtnStartScan_Click`):

1. **FRITZ!Box-Geräteliste** (optional): Lesen der Hosts per TR-064, bevor der Scan beginnt.
2. **Multicast-Vorabschritte** (mDNS, SSDP, WS-Discovery laufen parallel): Geräte, die nur auf diesem Weg gefunden werden (auch außerhalb des Scan-Bereichs), erscheinen sofort mit Status „Discovery“.
3. **Ping-Scan** des Bereichs, parallel (Auswahl 5, 10, 20 oder 50; Timeout 100–2000 ms). Je erreichbarer IP (`ScanHostAsync`):
   - **Ohne Ping-Antwort** (Option „Ohne Ping-Antwort“, Standard an): Im eigenen Subnetz löst der Ping eine ARP-Anfrage aus; antwortet die Station darauf, gilt sie als online, ein zweiter Ping fängt Geräte im WLAN-Stromsparmodus. In gerouteten Netzen prüft ein kurzer TCP-Check die Ports 445, 135, 80, 443, 22, 3389 und 62078 (auch ein abgewiesener Verbindungsaufbau zählt als Antwort). Solche Geräte stehen als „Online“ mit Antwortzeit „-“ und dem Dienst-Hinweis „kein Ping (ARP)“ bzw. „kein Ping (TCP …)“.
   - **System-Hinweis** aus der TTL der Antwort (64: Linux/macOS/Mobil, 128: Windows, 255: Netzwerkgerät), Spalte „System“.
   - **MAC-Adresse** (aus der Nachbartabelle bzw. per `SendARP`) und **Hersteller** (eingebettete Datenbank `mac-vendors-export.json`; es werden keine MAC-Adressen an externe Dienste gesendet).
   - **Hostname:** zuerst DNS-Reverse-Lookup, danach als Rückfallstufen LLMNR, mDNS, NetBIOS und ARP-Cache.
   - **SMB (Port 445)** nur bei aktivem „Banner-Grabbing“: Rechnername, Domäne und Windows-Build ohne Zugangsdaten. Die Anzeige „Windows“ oder „Samba“ wird erst am Scan-Ende entschieden (`FinalizeSmbInfo`), weil ein antwortender SMB-Dienst kein Windows beweist (FRITZ!Box, NAS, Linux sprechen dasselbe Protokoll).
   - **SNMP-Probe** (v2c oder v3), wenn aktiviert: liefert Gerätetyp, Namen und Daten für die Topologie.
   - **Switch-Verbindungen** (`SwitchLinkResolver.cs`): Nur direkte Links (LLDP, UniFi-Controller-Uplinks, sonst Vergleich der MAC-Tabellen: Switch C liegt zwischen A und B, wenn er beide auf verschiedenen Ports sieht oder A und B ihn je Richtung des anderen sehen), zu einem Baum ohne Schleifen zusammengeführt; Ergebnis in `NetworkDeviceVertex.SwitchLinks`, Wurzel am Gateway über `GatewayPort` bzw. `GatewayDirect`.
   - **Ersatz- und Einzelzugang** (`DeviceScanDialog.SnmpAccess.cs`): Ohne Antwort wird das Gerät einmal mit der anderen SNMP-Version wiederholt (Option „Bei keiner Antwort …“); der Zugang, der geantwortet hat, steht in `_snmpServiceByIp` und gilt auch für LLDP, Bridge-MIB, ARP und STP der Topologie-Analyse (`SnmpFor(ip)`). Kontextmenü „SNMP-Zugang für dieses Gerät …“ legt je IP einen Einzelzugang an (nur im Arbeitsspeicher, nicht gespeichert).
   - **Gerätetyp:** SNMP-Ergebnis zuerst, sonst Fingerbank (optional), sonst lokale Heuristik.

  - **Banner-Grabbing** (Option, Standard an) läuft parallel zum Ping-Scan; höchstens 20 Geräte gleichzeitig (je Gerät mehrere Ports, `SemaphoreSlim`).
1. **Nachbearbeitung:** mDNS-, SSDP- und WSD-Ergebnisse werden auf die Online-Geräte übertragen (Namen, Dienste, Gerätetyp, Apple-Modell); danach FRITZ!Box-Abgleich (Namen, MAC, „LAN/WLAN (FRITZ!Box)“, Geräte nur der Box, bei „Offline-Geräte“ auch deren Offline-Geräte), Vervollständigung der nur per Multicast gefundenen Geräte (MAC, Hersteller, Typ) und `FinalizeSmbInfo`.
2. **Abschlussmeldung** mit Anzahl der Online-Geräte (davon ohne Ping-Antwort); die Schaltfläche „Topologie“ wird freigegeben.

Scan-Optionen (ein-/ausblendbar, „Einstellungen beim Scan automatisch ausblenden“): Hostnamen (DNS), MAC-Adressen, Hersteller, Gerätetyp, Offline-Geräte, Fingerbank-API, Banner-Grabbing, Ohne Ping-Antwort, SNMP-Erkennung (Community bei v2c; Benutzer, Auth- und Priv-Protokoll und -Passwort, Context bei v3; eigener SNMP-Timeout), UniFi-API, FRITZ!Box-Geräteliste. Die Felder „Adresse/Benutzer“ von UniFi und FRITZ!Box werden gespeichert, Passwörter nicht.

Erkennung ohne Ping und Banner (Vorab-Schritte und Option „Banner-Grabbing“):

- **mDNS:** Neben `ANY local.` werden das Diensteverzeichnis (`_services._dns-sd._udp.local`), die PTR-Einträge wichtiger Diensttypen und `_device-info._tcp.local` abgefragt. Aus den Diensten wird der Gerätetyp abgeleitet (Drucker, Chromecast, Apple TV, HomeKit, NAS); bei Apple-Geräten liefert `_device-info` das Modell.
- **SSDP und WS-Discovery** (UDP 3702): erkennt Windows-PCs mit Netzwerkerkennung, WSD-Drucker/-Scanner und ONVIF-Kameras, auch außerhalb des Ping-Bereichs.
- **Banner:** HTTP (`Server`-Header, `<title>`, Realm), HTTPS (Zertifikatsname und Aussteller), SSH und FTP. Die Banner liefern einen Typ-Hinweis (FRITZ!Box, Drucker, Synology, QNAP, UniFi, MikroTik, Cisco, pfSense/OPNsense, Kameras, Proxmox/ESXi, IIS), der nur übernommen wird, wenn der Typ noch unbekannt ist.
- **SMB (Port 445):** SMB2-Negotiate und NTLMSSP-Negotiate ohne Zugangsdaten liefern Rechnername, Domäne und Windows-Build. Es findet keine Anmeldung statt, im Zielsystem entsteht kein fehlgeschlagener Versuch. Grenze: Ab Windows 11 24H2 / Server 2025 meldet Windows den Build nicht mehr über diesen Weg; der Wert bleibt bei 26100.

Geräteerkennung (Reihenfolge nach Beweiskraft): Methode `DetectDeviceType`.

1. Infrastruktur-Präfixe im Namen (`sw-`, `prn-`, `ap-`, `cam-`, `fw-`, `gw-`, `ups-` …)
2. eigene Muster (Feld „Eigene Computer-Namen“: Präfixe wie `ws-` oder Domänen-Endungen wie `.firma.local`; nur lokal gespeichert, Vorgabe leer)
3. allgemeine Computer-Namen (DESKTOP-…, LAPTOP-…, WIN-…)
4. Hersteller- und Namensregeln
5. zuletzt „Hostname mit Domäne → Computer (vermutlich)“

Vorrangig gilt ein SNMP-Treffer. Fingerbank (`FingerbankApiService`, API-Schlüssel nötig) ist optional; liefert sie nichts oder tritt ein Fehler auf, greift die lokale Heuristik (`LocalDeviceIdentificationService`). Es gibt keine fest eingebauten Firmen-Namensmuster.

Zusatzquellen:

- **UniFi-API:** Switch-Port-Zuordnung (MAC → Switch/Port) direkt vom Controller, Sites ladbar.
- **FRITZ!Box (TR-064):** nur lesend, HTTP-Digest; Standardadresse `http://fritz.box:49000`. Ist die Box nicht erreichbar oder lehnt sie die Anmeldung ab, steht ein Hinweis in der Statuszeile und der Scan läuft weiter.

Ergebnisliste und Aktionen: Spalten u. a. Status, IP-Adresse, Hostname, MAC-Adresse, Hersteller, Gerätetyp, System, Dienste, Antwortzeit. Suche mit „Weitersuchen“. Kontextmenü: IP/MAC kopieren, Ping, Traceroute, Port-Scan und IP & DNS Analyse (`NavigateToSidebarRequested`), „Öffnen mit …“ (Remotedesktop, SSH über Windows Terminal (`wt.exe`, OpenSSH-Client), Webbrowser HTTP/HTTPS, Computerverwaltung) und Fingerbank-Lookup. Die Schaltfläche „Topologie“ erstellt aus LLDP, Bridge-MIB,

ARP-Tabellen und STP eine Netzwerktopologie (`NetworkTopologyDialog`; erfordert SNMP-fähige Switches). Export als PDF, CSV, JSON und Text; Ergebnisse stehen in einer `ObservableCollection<NetworkDeviceInfo>`. Die Liste hat über sich Kacheln (Anzahl je Gerätetyp, Klick filtert), ein Suchfeld und unter sich einen Detailbereich (`DeviceScanDialog.Results.cs`). Die Topologie wird ohne Grafikpaket selbst angeordnet und gezeichnet (`NetworkTopologyDialog.Canvas.cs`: Baum Gateway → Switches mit den Geräten als Gruppe; `.Details.cs`: Kacheln, Auswahl, Detailbereich; `.Export.cs`: PNG, PDF).

5.2 Ping & Latenz (PingDialog)

Klasse: `PingDialog` (Dialogs/NetworkDiagnostics/), Ziele-Verwaltung in `PingDialog.Targets.cs` (`PingTarget`, `PingStats`) **Technologie:** `System.Net.NetworkInformation.Ping` (ICMP), `TcpClient` (TCP-Ping)

Misst Erreichbarkeit, Latenz, Jitter und Paketverlust. Features:

- **Mehrere Ziele** (höchstens 10, Tab „Ziele“) mit Übersichtstabelle je Ziel; Modus pro Ziel ICMP oder TCP (`host:port`)
- **Gateway mitprüfen** (immer ICMP) mit Auswertung „Gateway ↔ Ziele“: trennt Ursachen im lokalen Netz von solchen hinter dem Gateway
- **TCP-Ping** misst den Verbindungsaufbau zu einem Port (z. B. 443, 3389), nützlich bei gefiltertem ICMP
- **MTU-Erkennung** über Pakete mit gesetztem „Nicht fragmentieren“
- Freie Werte für Anzahl, Intervall, Timeout, Paketgröße (editierbare Auswahlfelder), DF-Bit und TTL
- **Verlustursachen** je Ziel (Zeitüberschreitung, Ziel/Netz nicht erreichbar, TTL abgelaufen, Paket zu groß, Verbindung abgelehnt …) mit Erklärung; meldendes Gerät wird genannt
- **Ausfall und Alarm:** Ausfalldauer, Anzahl Ausfälle, Schwellwerte für Verlust/Latenz, optional Alarmton
- Statistik: Min/Ø/Max, Median, 95 %-Perzentil, Jitter, Paketverlust, MOS-Schätzung (nur ICMP); Farbcodierung der Latenz (grün < 20 ms, blau < 80 ms, orange < 150 ms, rot ≥ 150 ms)
- Live-Diagramm, Verlauf als `ObservableCollection<PingHistoryEntry>`
- **Laufanzeige:** Laufbalken über die volle Breite an der Oberkante der Toolbar; Anzahl Ziele und Paket erscheinen nur während des Laufs als Badge in der Titelzeile
- **Export:** PDF (Zusammenfassung, Ziele-Tabelle, Auswertung, Verlustursachen; `InventoryPdfExporter.ExportReport`), CSV (alle Pakete) und TXT
- `SetHostname(string)` startet den Ping direkt – wird von anderen Dialogen genutzt; Enter im Eingabefeld startet/stoppt

5.3 Traceroute (TracerouteDialog)

Klasse: `TracerouteDialog` (Dialogs/NetworkDiagnostics/) **Technologie:** `System.Net.NetworkInformation.Ping` mit TTL-Manipulation; TCP-Verfahren über `Socket` und einen ICMP-Raw-Socket (`IcmpListener`) **Dateien:** `TracerouteDialog.xaml.cs` plus Partials `.Analysis`, `.Tcp`, `.Operators`, `.Chart`

Verfolgt den Paketweg zum Zielhost hop-by-hop und misst ihn danach wie MTR wiederholt:

- **Ermittlung:** ICMP-Pakete mit steigendem TTL; Reverse-DNS je Hop. Optionen: maximale Hops (1–255), Timeout je Probe (200 ms–30 s, Standard 1,5 s), Paketgröße (0–65500 Bytes, nicht fragmentiert – deckt MTU-Engstellen auf), IP-Version (Auto/IPv4/IPv6)
- **MTR-Messung:** Felder „Proben“ und „Abstand“; je Hop Verlust, Min/Ø/Max und Jitter über alle Runden (`HopEntry`, `ProbeResult`)
- **Bewertung** (`TraceAnalysis.Assess/BuildVerdict`): Verlust oder Verzögerung auf einem Zwischen-Hop gilt nur dann als Fehler, wenn er sich bis zum Ziel fortsetzt (sonst Rate-Limiting des Routers).

Ergebnis: Bewertung je Hop und Gesamtbefund (`TraceVerdict`) mit Netzgrenze, größtem Latenzsprung und Betreiberwechsel

- **TCP-Traceroute:** TCP-Verbindungsaufbau zu einem Port mit steigender TTL, passiert auch Firewalls, die ICMP sperren. Der Port muss auf dem Ziel offen sein. Zwischen-Hops werden nur mit Administratorrechten erkannt (Raw-Socket); sonst zeigt eine Hinweisleiste den Grund. Fängt ein transparenter Proxy die Verbindung ab, warnt der Befund
- **Betreiber (AS):** Spalte „Betreiber (AS)“ je Hop über DNS-Abfragen bei Team Cymru; es werden nur Router-Adressen als DNS-Namen gesendet. Schaltbar (Chip „Betreiber (AS)“, Stufe „Netz- und Betreiberzuordnung“ der zentralen Einwilligung `ExtLookupNetworkInfo`, Standard aus)
- **Streckengrafik** (Tab „Grafik“): Ø-Latenz je Hop als Punkt mit Min–Max-Balken, Verlust als roter Balken, × für stille Hops, ▲ bei Latenzsprung, Hintergrundbänder je Betreiber; wächst live und wird aus der Historie wiederhergestellt
- **Laufanzeige:** Laufbalken über die volle Breite an der Oberkante der Toolbar; Hop-Zähler bzw. Messrunde erscheinen nur während des Laufs als Badge in der Titelzeile
- **Export:** PDF (Zusammenfassung, Befund, Hop-Tabelle), CSV und TXT; Historie früherer Läufe

5.4 IP & DNS Analyse (DnsLookupDialog)

Klasse: `DnsLookupDialog` (Dialogs/NetworkDiagnostics/), Logik in `DnsLookupService` (Partial-Dateien `.Checks`, `.Dnssec`, `.Servfail`, `.Services`) **Technologie:** `DnsClient` NuGet-Paket, `System.Security.Cryptography` (DNSSEC-Prüfung)

Der Dialog hieß früher „DNS-Auflösung“ bzw. „DNS-Analyse“. Er nimmt einen Hostnamen oder eine IP-Adresse entgegen und wertet sie in zwei Phasen aus. Phase 1 liefert die Records, Phase 2 die Zusatzprüfungen; die Karten füllen sich nacheinander.

Records und Grundprüfungen

- Typen: A, AAAA, CNAME, NS, SOA, MX, TXT, SRV, PTR; Reverse-Validierung (PTR zeigt zurück?)
- DNSSEC und CAA (vom Namen aufwärts bis zur Zone), MTA-STS und TLS-RPT, SPF mit Zählung der DNS-Abfragen (max. 10, rekursiv), DMARC
- Propagation gegen mehrere öffentliche Resolver. Der gewählte Resolver steht als erste Zeile; Adressen, die nicht in der autoritativen Antwort vorkommen, sind markiert.
- Antwortzeit des gewählten Servers, zusammenfassende **Bewertung** (Fehler rot, Warnungen orange, Empfehlungen blau; Klick springt zur Karte) und **Vergleich mit der letzten Abfrage** desselben Hosts
- Abfrage-Verlauf als `ObservableCollection<DnsHistoryEntry>`, Direktstart via `SetHostname(string)`

Zusatzprüfungen (Karten)

Karte	Inhalt
Autoritative Server	Jeder Nameserver der Zone wird ohne Rekursion direkt befragt: SOA-Seriennummer (Vergleich nur innerhalb eines DNS-Anbieters, erkannt am SOA-MNAME), AA-Flag, Antwortzeit, Adresse und TTL. Meldet: nur ein Nameserver, alle im selben /24, nicht auflösbare oder nicht antwortende Server, Lame Delegation, REFUSED, abweichende Seriennummern. IPv6-Server werden nur bei vorhandenem IPv6 befragt.
Verweise & Namen	CNAME-Kette bis zum Ziel: nicht existierendes Ziel (NXDOMAIN), Hinweis auf mögliche Subdomain-Übernahme bei bekannten Anbietern (Azure, GitHub Pages, Heroku, S3, CloudFront u. a.), Schleifen, sehr lange Ketten; MX-Ziele ohne Adresse, als CNAME oder als IP-Literal. Läuft auch bei NXDOMAIN.
Sicherheit (DNSSEC)	DNSKEY der Zone, Abgleich des DS-Eintrags beim Parent mit den Schlüsseln (Schlüssel-ID, Prüfsumme SHA-1/256/384), Algorithmus, Laufzeit der Signaturen. Fehler: DS ohne DNSKEY, DS passt zu keinem Schlüssel, abgelaufene Signaturen. Warnungen: SHA-1-Algorithmen, Signaturen, die bei langem Gültigkeitsfenster in unter 3 Tagen ablaufen.

Karte	Inhalt
SERVFAIL-Diagnose	Nur bei SERVFAIL des gewählten Resolvers: Der Name wird ab den Root-Servern ohne Validierung selbst aufgelöst und die DNSSEC-Kette der Zone geprüft (DS ↔ DNSKEY, Signatur des DNSKEY-Satzes, Signatur der A-Antwort kryptografisch mit RSA und ECDSA, Gültigkeitsfenster). Die Karte nennt die Ursache und vergleicht mit den öffentlichen Resolvern. Nicht geprüft: Kette oberhalb der Zone, NSEC/NSEC3, Ed25519/Ed448. Ohne ausgehenden UDP/53-Verkehr nicht möglich.
Erkannte Dienste	Indizien aus MX (Mailanbieter), SPF-Includes (Versanddienste), CNAME-Proben (Autodiscover, Geräteregistrierung, DKIM-Selektoren von Microsoft 365), Google-DKIM, DMARC-`rua=` und TXT-Bestätigungen (`MS=`, `google-site-verification` u. a.; unbekannte nur mit dem Schlüssel, nie mit dem Wert). Aufräum-Hinweise: SPF und MX nennen verschiedene Plattformen, Skype-for-Business-Reste, ab 10 Bestätigungs-Einträgen. Das sind Hinweise, kein Nachweis.
IP-Informationen	Reverse-DNS, ASN, Netz (BGP-Präfix) und Registry über die DNS-Zonen von Team Cymru (`origin.asn.cymru.com`, ohne Schlüssel); ergänzt und ersetzt bei Ausfall ipinfo.io. Nur bei eingeschaltetem „IP-Info online“ (Standard aus; Einwilligung `ExtLookupNetworkInfo`); ohne den Schalter verlässt keine IP den Rechner.

Weitere Auswertungen: **TTL** (in der Zone eingestellt vs. Restlaufzeit beim Resolver; Hinweis unter 60 s und ab 24 h) und **Wildcard** (zufälliger Name der Zone; meldet auch einen Resolver, der Fehlerseiten umleitet).

Große Antworten: Ist die UDP-Antwort abgeschnitten (große TXT-Sätze, DNSKEY), weicht der Dialog mit kurzem Timeout auf TCP aus (`QueryLargeAsync`) und merkt sich einen gesperrten TCP/53 für zwei Minuten. Bei unvollständiger Antwort steht ein Hinweis, die Aussage „kein SPF“ entfällt.

Export: CSV, TXT und PDF (`IP-DNS-Analyse_<Abfrage>`), jeweils mit allen Karten. Schnellaktionen je Adresse: Ping, Traceroute, Port-Scan; für Mail-Domains zusätzlich „Domain-Analyse“.

5.5 Port-Scan (PortScanDialog)

Klasse: `PortScanDialog` (Dialogs/NetworkDiagnostics/) **Technologie:** `System.Net.Sockets.TcpClient/UdpClient` (async); aufgeteilt in Partials `.Catalog`, `.Targets`, `.Probes`, `.Engine`

Scannt TCP- und UDP-Ports auf Erreichbarkeit:

- **Ziele:** mehrere Hosts, Bereiche und Netze, höchstens 256 Ziele je Lauf
- **Auswahl-Leiste:** Vorlagen, Dienste (Popup mit Chips und Gruppen-Checkbox; SMTP ist nicht vorausgewählt), freie Ports/Bereiche, Modus (TCP/UDP) und Optionen
- **UDP:** dienstspezifische Proben (z. B. DNS auf Port 53); ohne Antwort „KEINE ANTWORT“
- **Banner-/Zertifikat-Abfrage:** standardmäßig aus (Opt-in)
- Status: OFFEN, GEFILTERT (Timeout), GESCHLOSSEN (Verbindung abgelehnt), KEINE ANTWORT (UDP)
- Servicename-Zuordnung, Statistik-Kacheln als Filter, Suchfeld und Chip „Nur offene“, Matrix-Ansicht
- „Unklare erneut“ prüft nur gefilterte bzw. unbeantwortete Ports erneut; geänderte Status werden markiert
- Laufbalken über die volle Breite unter der Titelzeile
- Datenmodell: `PortResult`; Scan-Verlauf `ScanHistoryEntry` mit Wiederherstellung
- Export: PDF (`InventoryPdfExporter.ExportReport`), CSV, TXT

5.6 Netzwerk & Verbindungen (NetzwerkHubDialog)

Klasse: `NetzwerkHubDialog` (Dialogs/NetworkDiagnostics/)

Bündelt mehrere Netzwerk-Werkzeuge als Registerkarten: **Netzwerkkarten**, **Routing-Tabelle**, **Verbindungen**, **Netzwerk-Analyse**, **Netzwerklast** und **Diagnose**. Die Schaltfläche **Gesamtbericht** (alle Reiter; „Exportieren“ in der Titelzeile betrifft nur den aktuellen Reiter) erzeugt einen Gesamtbericht als PDF oder Text (`NetworkReport`): Überblick mit der Karte der Standardroute, Netzwerkkarten mit Details, Routing mit Bewertung, Verbindungen (auch zu öffentlichen Adressen) und – sofern im Reiter gelaufen – Netzwerklast und Diagnose mit Gesamtbewertung. Karten, Routing und Verbindungen werden dafür frisch erhoben. Aus dem System-Health-Dialog kann direkt die Registerkarte Netzwerkkarten geöffnet werden (`ShowNetzwerkkarten()`).

Netzwerkkarten (NetworkCardsDialog)

Klasse: `NetworkCardsDialog` (Dialogs/NetworkDiagnostics/) **Technologie:** `System.Net.NetworkInformation.NetworkInterface`, WMI (`MSFT_NetConnectionProfile`, `Win32_NetworkAdapterConfiguration`), `netsh wlan show interfaces`

Listet alle Netzwerkadapter des Systems auf und zeigt Detailinformationen:

Linke Spalte (Adapterliste):

- Filtermöglichkeit: Alle / Online / Ethernet / WLAN / VPN-Tunnel
- Adapter-Karte mit Icon, Name, IP-Adresse, Online/Offline-Status, Geschwindigkeit
- Die **Karte der Standardroute** (über sie leitet Windows den Verkehr ins Internet) steht oben, ist beim ersten Öffnen ausgewählt und trägt den Vermerk „Standardroute“. Ermittelt wird sie mit `GetBestInterface` zu einer öffentlichen Adresse (`PrimaryRoute`), ohne etwas zu senden.

Rechte Spalte (Detailansicht):

- Allgemeine Infos: Typ, MAC-Adresse, Geschwindigkeit, MTU, Netzwerk-ID, Metrik, Treiber (Version, Datum, Signatur), Wake-on-LAN, NIC-Teaming
- **Netzwerkprofil:** Name, Kategorie (Öffentlich, Privat, Domäne) und Konnektivität (Internetzugang, nur lokales Netz …)
- **WLAN-Verbindung** (bei WLAN-Karten): SSID, Access Point (BSSID), Signal mit Einordnung, Funktyp/Band/Kanal, Raten, Authentifizierung und Verschlüsselung (deutsche und englische Ausgabe von `netsh` werden gelesen)
- IPv4-Konfiguration: IP, Subnetzmaske, Gateway, DNS-Server, DNS-Suchsuffix, WINS, DHCP-Lease
- DHCP-Badge (DHCP / Statisch) mit Server-Anzeige
- IP erneuern (ipconfig /release + /renew) für DHCP-Adapter
- **Gateway anpingen:** Öffnet den Ping-Dialog mit der Gateway-IP (via `NavigateToSidebarRequested`)
- IPv6-Adressen (falls vorhanden)
- Netzwerk-Statistik: Gesendet, Empfangen, Fehler, Verworfene Pakete
- Schnellaktionen: IP kopieren, MAC kopieren, Alles als Text kopieren (enthält Treiber, Profil und WLAN-Angaben). Die Rückmeldung erscheint in der Kopfzeile, nicht als Dialog.

Routing-Tabelle (RoutingDialog)

Klasse: `RoutingDialog`, `RoutingModel` (Dialogs/NetworkDiagnostics/) **Technologie:** WMI (`MSFT_NetRoute`, `MSFT_NetIPInterface`)

Zeigt aktive und persistente Routen (IPv4/IPv6) mit Bewertung:

- Spalten: Ziel, Gateway, Adapter, wirksame Metrik (Route + Schnittstelle), Herkunft, Art (aktiv/persistent), Hinweis
- Die **wirksame Standardroute** ist mit „★“ markiert und im Hinweis als „Wirksame Standardroute“ benannt (bei IPv4 die Route der Karte, über die Windows Internetverkehr tatsächlich leitet, sonst die mit der kleinsten Metrik); die Kachel „Default-Gateway“ zeigt sie

- Bewertung: mehrere Default-Routen (mit gleicher Metrik als Warnung), APIPA-Gateway, Gateway außerhalb des eigenen Netzes, dasselbe Netz auf mehreren Adaptern (z. B. VPN und LAN), persistente Routen ohne aktiven Adapter, manuell eingetragene Routen; ein Klick auf einen Befund filtert die betroffenen Routen
- Routen hinzufügen und gezielt löschen (nur mit Administratorrechten, mit Befehlsvorschau); Export als PDF und TXT

Verbindungen (ConnectionsDialog)

Klasse: `ConnectionsDialog` (Dialogs/NetworkDiagnostics/), enthält auch die Reiter **Netzwerk-Analyse** und **Netzwerklast Technologie:** `GetExtendedTcpTable` / `GetExtendedUdpTable` (iphlpapi, IPv4 und IPv6)

Zeigt alle TCP/UDP-Verbindungen des Systems mit Prozess:

- Auto-Aktualisierung (5 s), Statistik-Badges (Gesamt, Established, Listening, TCP, UDP) als Filter, Suche, Top-Prozesse (Klick filtert) und Verbindungshistorie mit Snapshots
- Filter: Alle, nur TCP, nur UDP, nur ESTABLISHED, nur LISTENING, **nur extern** (öffentliche Remote-Adresse), **neu seit letzter Aktualisierung**, **beendet seit letzter Aktualisierung**. Die Spalte vor „Protokoll“ markiert neue Verbindungen mit „neu“, die Statuszeile nennt „n neu, m beendet“. Wiedererkannt wird eine Verbindung an Protokoll, lokaler und entfernter Adresse und Prozess.
- Anreicherung (standardmäßig aus, zwei Stufen der zentralen Einwilligung, getrennt schaltbar über die Menüpunkte „Online: Land“ und „Online: Reputation“): Land (ip-api.com, `ExtLookupNetworkInfo`), Reputation (Shodan InternetDB, `ExtLookupReputation`), Authenticode-Signatur des Prozesses, Risikobewertung; Recherche-Links zu VirusTotal, AbuseIPDB u. a. im Kontextmenü
- Prozess-ID: eigene Spalte „PID“ in der Tabelle, außerdem Prozess-ID, Prozessname und Pfad im Detailbereich
- **Prozess beenden:** Schaltfläche im Detailbereich (Zeile „Prozess-Check“) und Kontextmenü-Eintrag „Prozess beenden …“ beenden den zur gewählten Verbindung gehörenden Prozess (`Process.Kill`, ohne Kindprozesse). Vorher erscheint eine Sicherheitsabfrage mit Name, PID und Zahl der zugeordneten Verbindungen (voreingestellt „Nein“). Nicht möglich sind System/Leerlauf (PID ≤ 4), bereits beendete Verbindungen und die Diagnose-Anwendung selbst; geschützte Prozesse erfordern Administratorrechte. Danach wird die Liste aktualisiert.

Netzwerk-Analyse (Reiter im ConnectionsDialog)

Prüft die Netzwerkkonfiguration und misst die Qualität in zwei Phasen: zuerst die Konfiguration (TCP-Einstellungen, Verbindungen, DHCP, ARP, NetBIOS/LLMNR, parallel), danach allein die Messungen (MTU, DNS, Bufferbloat). Auffälligkeiten erhalten eine Bewertung mit Empfehlung; die Auswertung läuft im Hintergrund.

- **TCP-Einstellungen:** per WMI (`MSFT_NetTCPSetting`, Vorlage „Internet“, und `MSFT_NetOffloadGlobalSetting`, Namespace `root\StandardCimv2`) statt Textauswertung von `netsh`; sprachunabhängig und ohne Prozessstart. Bewertet werden u. a. Auto-Tuning, Timestamps und der Congestion Provider (BBR/CUBIC/DCTCP/NewReno/LEDBAT unauffällig, bei CTCP der Hinweis `template=internet congestionprovider=cubic`).
- **Port-Auslastung:** Gezählt werden nur ausgehende TCP-Verbindungen, jeder Port einmal, innerhalb des tatsächlichen dynamischen Portbereichs (Start und Anzahl); die Empfehlung nennt den echten Startport. TIME_WAIT wird aus der Verbindungstabelle gezählt.
- **DHCP-Lease:** nativ aus `NetworkInterface` und WMI (`Win32_NetworkAdapterConfiguration`, Zuordnung über den Schnittstellenindex). Erkannt wird auch „DHCP aktiv, aber kein Lease“ (APIPA 169.254.x.x).
- **IPv6-Hinweis:** Es zählen nur bestehende Verbindungen (ESTABLISHED) zu echten IPv6-Gegenstellen, ohne Loopback und Link-Local.

- **ARP-Cache:** auf Basis der Windows-Nachbartabelle (`GetIpNetTable2`, `Services\NeighborTable.cs`, `ReadAllIPv4`). Echte IP-Konflikte sind im ARP-Cache nicht erkennbar (Windows hält je Schnittstelle nur einen Eintrag pro IP), die Prüfung entfällt; geprüft wird das ARP-Spoofing-Muster (Gateway-MAC wird auf einem anderen Gerät gesehen).
- **NetBIOS/LLMNR:** Geprüft werden nur aktive Adapter; betroffene Adapter werden genannt.
- **MTU:** Zuerst Erreichbarkeit mit kleinem Paket (8.8.8.8, sonst 1.1.1.1); Timeouts werden einmal wiederholt. Ist kein Ziel per ICMP erreichbar, erscheint ein eigener Hinweis statt eines falschen Werts.
- **DNS-Latenz:** direkte Abfrage beim DNS-Server über `DnsClient` (ohne Cache, ohne Wiederholung, 2 s Timeout) mit einer Aufwärm-Abfrage, die nicht mitzählt; zusätzlich drei zufällige, nicht gecachte Namen für die vollständige rekursive Auflösung (Zusatzwert im Detail). Timeouts zählen nicht in den Durchschnitt, sondern werden genannt.
- **Bufferbloat:** je 5 s Download-Last (4 Streams, höchstens 250 MB) und Upload-Last (2 Streams, höchstens 50 MB) gegen `speed.cloudflare.com`; Latenz und Durchsatz je Richtung im Detail, Bewertung nach der schlechteren Richtung.

Netzwerklast (Reiter im ConnectionsDialog)

Live-Messung alle 2 Sekunden für einen gewählten Adapter (vorgewählt ist die Karte der Standardroute): Eingehend, Ausgehend, Fehler, Verlauf der letzten 15 Minuten als Diagramm, Mittelwert und Spitzenwert mit Uhrzeit, Hinweise bei Dauerlast oder Fehlern.

Prozesse nach Netzwerklast: Eine Tabelle zeigt, welche Prozesse die Last verursachen (aktuell Empfang und Senden, Verbindungen, Summe seit Start). Grundlage ist die TCP-Verbindungsstatistik von Windows (`SetPerTcpConnectionEStats` / `GetPerTcpConnectionEStats`, Typ „Data“; Klasse `ProcessNetworkLoad`), zugeordnet über die Verbindungstabelle. Voraussetzungen und Grenzen: **nur mit Administratorrechten** (sonst erscheint ein Hinweis), nur TCP über alle Adapter (nicht je Adapter; UDP/QUIC und lokale Verbindungen fehlen), gezählt wird ab dem Einschalten der Statistik (schon laufende Verbindungen liefern ab der zweiten Messung Werte, kurze Übertragungen vor der ersten Messung fehlen). Beim Stoppen wird die Statistik wieder ausgeschaltet.

Diagnose (NetzwerkDiagnoseDialog)

Acht Prüfungen nacheinander: Gateway (Ping mit Jitter und Paketverlust, Standardroute zuerst), DNS, Internet-Konnektivität, Proxy/WPAD, NCSI, Firewall (Outbound-Blockregeln), ARP-Cache, NetBIOS/LLMNR. Die **DNS-Prüfung** fragt jeden Server einzeln direkt ab (die DNS-Server der Karten, dazu 8.8.8.8 und 1.1.1.1) und zeigt Antwortzeit und Ergebnis; ein ausgefallener eigener DNS-Server wird als Fehler gemeldet, nicht erreichbare öffentliche Server nur als Hinweis (in Firmennetzen üblich).

Oben steht eine **Gesamtbewertung**, die der Kette Gateway → DNS → Internet folgt und sagt, wo das Problem liegt: lokales Netz (Gateway nicht erreichbar), hinter dem Router (weder DNS noch Internet), DNS, blockierte Ziele (Proxy, Firewall, Captive Portal) oder „Netzwerk unauffällig“ (Ursache dann eher Anwendung, Konto oder Zielserver). Der Berichtsexport des Hubs enthält die Gesamtbewertung.

5.7 DHCP-Discovery (DhcpDiscoveryDialog)

Klasse: `DhcpDiscoveryDialog` (Dialogs/NetworkDiagnostics/) **Technologie:** UDP-Sockets (Port 67/68 für DHCPv4, 546/547 für DHCPv6), Windows-Nachbartabelle (`NeighborTable`, `GetIpNetTable2`)

Findet alle DHCP-Server im Netzwerk und bewertet, ob ein unerwünschter Server (Rogue-DHCP) antwortet. Erfordert Administratorrechte; ohne sie bietet der Dialog den Neustart über `App.RestartAsAdministrator()` an.

Ablauf:

1. **Ausgangslage festhalten:** Adapterkonfiguration (IP/Maske, Gateway, DNS, DHCP an/aus) und der in der Registry gespeicherte DHCP-Server (nur ausgewertet, wenn `EnableDHCP = 1`).

2. **Aktive Suche:** DHCPv4-DISCOVER und DHCPv6-SOLICIT (RFC 8415: DUID-LLT mit vollständiger MAC, Elapsed Time, IA_NA) über den gewählten Adapter. Wartezeit 5 s, Wiederholung nach 2 s, ein gemeinsamer Countdown. Fehlerhafte oder abgeschnittene Pakete werden übersprungen (Grenzprüfung je Option, Empfangspuffer 4096 Bytes).
3. **Anreicherung nach der Wartezeit**, für alle Server parallel: Hostname (Reverse-DNS), MAC (aus der Nachbartabelle, bei Bedarf löst ein kurzer Ping die ARP-/NDP-Auflösung aus) und Hersteller (zentrale Hersteller-Datenbank `LocalDeviceIdentificationService`, offline).
4. **Gruppierung** nach Gerät (`DhcpServerGroup`): Ein Dual-Stack-Server mit DHCPv4 und DHCPv6 erscheint als eine Karte. Bei DHCPv6 liefert die Server-DUID (LLT/LL, Ethernet) die MAC des Servers, wenn der Nachbar-Cache sie nicht kennt (Kennzeichnung „aus Server-DUID“).
5. **Bewertung** und, falls aktiviert, **passive Überwachung**.

Client-Kennung: Die Option „Lokal verwaltete MAC verwenden (empfohlen)“ (Standard an, Einstellung `DhcpUseRandomMac`) sendet pro Suche eine zufällige, lokal verwaltete MAC und den Hostnamen „DATEXT-Scanner“. Die eigene IP-Vergabe und der Gerätename dieses PCs im Router bleiben unberührt. Aus: echte Adapter-MAC mit dem eigenen Rechnernamen (nötig bei MAC-Filter oder DHCP-Snooping). Antwortet im Zufallsmodus kein Server, nennt der Hinweis diese Ursachen und die eingehenden Ports (UDP 68 / 546) als mögliche Firewall-Ursache.

Angebotsdaten (je Server): angebotene IP, Maske, Gateway, Lease-Zeit, DNS-Server (6), Domäne (15), NTP-Server (42), Domain-Suchliste (119), PXE (TFTP-Server 66, Bootdatei 67, `siaddr`, `file`), Hersteller-Optionen (43, Länge und Hex-Vorschau), Server-ID mit DUID, Relay (`giaddr`) und Absender. DHCPv6: DNS-Server (23), Domain-Suchliste (24).

Bewertung:

- **Pro Protokoll:** Mehrere DHCPv4- bzw. DHCPv6-Server auf verschiedenen Geräten = Warnung „Rogue-DHCP!“. DHCPv6 auf einem anderen Gerät als DHCPv4 ist nur ein Hinweis (häufig harmlos: DHCPv4 vom Windows-Server, DHCPv6 vom Router).
- **Abweichende Angebote** (Status „Abweichende Angebote“): Die Angebote werden untereinander und mit der Adapterkonfiguration verglichen. Unterschiedliche Gateways, DNS-Server, Subnetze oder Domänen verschiedener Server sind ein starkes Rogue-Indiz. Weicht der genutzte Server selbst ab, ist das ein Hinweis; bei statisch konfiguriertem Adapter gibt es nur Hinweise.
- **Relay und Absender:** `giaddr` gesetzt = Angebot über ein DHCP-Relay (Hinweis). Weicht der Absender ohne Relay von der Server-ID ab, oder kommen Angebote derselben Server-ID von verschiedenen MACs, ist das eine Warnung (Server mit mehreren Adressen oder gefälschte Server-ID). Eine Fälschung mit identischer IP ist nicht erkennbar.
- **Sicherheitsrelevante Optionen:** Klassenlose Routen (121, RFC 3442, und die Microsoft-Variante 249) und WPAD (252) werden angefragt. Routen, deren Bereich öffentliche Adressen umfasst, ergeben eine Warnung (TunnelVision, CVE-2024-3661: Die Routen sind spezifischer als die VPN-Route und leiten Verkehr am VPN vorbei). Geprüft wird der gesamte Bereich, nicht nur die Startadresse. WPAD ist ein Hinweis.
- **IPv6-Router (passiv):** Nach der Suche liest der Dialog die IPv6-Router des Adapters aus dem Windows-Zustand (Standard-Gateways, als Router markierte Nachbarn, per Router Advertisement gebildete SLAAC-Präfixe) – es werden keine Pakete gesendet. Mehrere IPv6-Router auf einem Adapter ergeben eine Warnung (ein fremdes Gerät kann per Router Advertisement auch in reinen IPv4-Netzen Gateway und Adressen verteilen); Hinweise gibt es, wenn der Router mit dem DHCP-Server identisch ist oder ein anderes Gerät als der DHCPv4-Server ist.

Passive Überwachung (Option „Danach passiv auf DHCP-Server achten für …“, 1 Minute bis höchstens 30 Tage, Dauer und Einheit werden gespeichert): Nach der aktiven Suche lauscht der Dialog auf UDP 68 auf OFFER und ACK (nur BOOTREPLY) und findet so Server, die nur zeitweise aktiv sind. Ein neuer Server wird sofort mit Zeitstempel gemeldet („Passiv gesehen“, Status „Neuer DHCP-Server!“) und neu bewertet. Ausgewertet werden nur Server-Angaben; Daten anderer Clients werden weder angezeigt noch protokolliert. Beenden jederzeit mit Stopp.

Oberfläche und Export: Laufbalken über die volle Breite an der Oberkante der Toolbar, Status als Badge in der Titelzeile. Live-Log der DISCOVER/OFFER-Kommunikation, Statuskacheln (Adapter, Geräte, Dauer). Export als PDF (`InventoryPdfExporter.ExportReport`), TXT und CSV (inklusive DHCPv6 und der neuen Angebotsspalten).

5.8 Firewall-Log-Auswertung (FirewallLogDialog)

Klassen: `FirewallLogDialog` (+ `.Export.cs`, Dialogs/NetworkDiagnostics/), `FirewallLogParser` und `FirewallLogAnalysis` (Services/FirewallLogAnalysis.cs), `CymruAsnService` (Services/) **Technologie:** Regex-Parser, DnsClient (Team Cymru IP-to-ASN per DNS), `InventoryPdfExporter.ExportReport`

Wertet Firewall-Logs aus (Format der Securepoint UTM) und gruppiert die Quell-Adressen nach Betreiber (ASN) und Netz (BGP-Präfix). Parser, Auswertung und Abfrage sind von der Oberfläche getrennt und lassen sich offline testen. Eingelesen werden das Bildschirm-Format und das Syslog-Dateiformat (`yyyy-MM-ddTHH:mm:ss.fff+zz:zz Programm[PID]: Meldung`); Sitzungen und Fehlanmeldungen stehen dort als `msg='session opened'` bzw. `msg='failure'`, die doppelte `auth failure:`-Zeile wird über Adresse, Dienst, Konto und ± 2 s zusammengeführt (`DropDuplicates`). Weitere Formate liegen in `ServicesFirewallLogFormats.cs` (`TryLine`: sshd, fail2ban, Postfix, Dovecot, Schlüssel=Wert, CEF/LEEF; `TryWindows`: Ereignis 4625; pfSense/OPNsense: `TryFilterLog`, Web-Anmeldung `WebGuiAuth`; Suricata: `TrySuricata`); `FirewallLogParseResult.Format` nennt das erkannte Format. Zeilen ohne Zeitstempel am Anfang werden zeilenweise gelesen (`ParseLines`), findet keine Zeile ein Format, gilt die Eingabe als IP-Liste. Abgleich von Doppelmeldungen: `DropDuplicates`.

Parser (`FirewallLogParser.Parse`): Ein Eintrag beginnt mit einem Zeitstempel (`dd.MM.yyyy HH:mm:ss` oder ISO); die Felder danach (Kategorie, Stufe, Meldung) sind durch Zeilenumbruch oder Tabulator getrennt. Erkannt werden

- die Fail2Ban-Meldung `Die IP-Adresse "…" wurde für den Service "…" für "…" Sekunden gesperrt.` (Dienst und Sperrdauer werden gelesen, „für“ darf durch falsche Zeichenkodierung entstellt sein; als Ausweichmuster gilt eine Meldung mit IPS-/Sperr-Kategorie und einer IPv4-Adresse),
- Anmeldefehler `auth failure:[user=…][service=…][realm=…][remote=IP;Port]` (saslauthd). Die übrigen Zeilen desselben Versuchs (spauthd, `authentication:…`, DEBUG) tragen keine verlässliche Quelladresse und zählen als Begleitzeilen (`Merged`), alle anderen Einträge als übersprungen (`Skipped`).

IP-Liste: Findet der Parser keinen Zeitstempel, ruft er `ParseIpList` auf (`FromList = true`): Zerlegung an Leerraum, Komma, Semikolon, Klammern und Anführungszeichen, der Port wird abgeschnitten, nur gültige Adressen (IPv4 mit vier Teilen, IPv6) zählen; jede ergibt ein Ereignis der Art `Listed` mit `DateTime.MinValue`, das in der Zählung wie eine Sperre behandelt wird. Ein `/Präfix` macht die Eingabe zum Netz: `Mask` maskiert auf die Netzadresse, `FirewallEvent.Net` hält die CIDR-Schreibweise (`/32` bzw. `/128` = Einzeladresse, Längen außerhalb 1…32 bzw. 1…128 werden verworfen). `CountByIp` führt Netz und Einzeladresse mit gleicher Anfangsadresse getrennt (`FwIpStat.Net`, `ListEntry`); die Betreiber-Abfrage läuft über die Netzadresse, `NetNote` warnt, wenn das Eingabenetz größer ist als das angekündigte. `BuildBlockList` lässt Eingabenetze als Netz stehen, außer ein gesperrtes Netz deckt sie vollständig ab, und entfernt Einzeladressen, die ein aufgeführtes Netz einschließt (`InNet`).

Sitzungen: `SessionRx` zerlegt `authentication:<Quelle>[/<mid>/<VPN-IP>]session opened|closed<Rest>`; die Quelle wird mit `Normalize` bereinigt (`::ffff:10.1.9.99` → `10.1.9.99`). Der Rest beginnt mit dem Dienst (`openvpn`, `ssl-vpn`, `l2tp`, `ipsec`, `pptp`, `*-ui`), danach folgen Benutzer, bei VPN die schon bekannte VPN-Adresse und bei „closed“ die Dauer in Sekunden (`^(?<u>.+?)<VPN-IP>(?<d>\d*)$`). Ohne VPN-Adresse (Weboberfläche) hängt die Dauer am Benutzernamen: `ResolveSessionUsers` trennt sie über die Benutzer aus den „opened“-Zeilen (längster passender Name, Rest nur Ziffern), sonst gelten die Endziffern als Dauer. `BuildSessions` paart Ereignisse nach Benutzer, Dienst, Quelle und VPN-Adresse; ein „closed“ ohne „opened“ erhält `Start = Ende − Dauer` (`StartDerived`, `BeforeLog`), ein erneutes „opened“ derselben Verbindung ohne „closed“ markiert die ältere Sitzung als `EndMissing`, übrige gelten als `Running`. `BuildSessionUsers` leitet die Auffälligkeiten ab (Überschneidung mit verschiedener Quelle, mindestens 3

Sitzungen unter 60 s, Beginn 22–5 Uhr, mindestens 2 Länder); `CountByHour` ergänzt `SessionStarts` und `MaxParallel` (Zeitachse abschreiten, Enden vor Anfängen). `AdFailRx` liest „<Konto>: AD authentication failed“ als Anmeldefehler ohne Quelladresse (`Ip` leer); `DropDuplicateAdFailures` entfernt ihn, wenn ± 2 s dasselbe Konto mit Quelladresse (saslauthd) vorliegt. Der Dialog nimmt die Quelladressen der Sitzungen in die Team-Cymru-Abfrage auf, die Sperrliste verwendet sie nicht (`CountByIp` zählt nur Sperrungen und Anmeldefehler).

Auswertung (`FirewallLogAnalysis`): Zählung je IP, Konto und Stunde (`CountByIp`, `CountByUser`, `CountByHour`); Gruppierung je ASN (`GroupByAsn`) und je Präfix (`GroupByPrefix`). Der Betreiber wird über Namensbestandteile eingestuft (`Classify`: Großanbieter, Hoster, Zugangsprovider, sonst unbekannt; kurze Wörter nur als eigenes Wort). `Recommend` leitet je Netz die Aktion ab: Großanbieter und Netze mit Präfixlänge unter /16 (IPv6 unter /32) nur Einzel-IPs; Hoster ab 2 Quellen Netz sperren; Zugangsprovider ab 3, unbekannte Betreiber ab 2 Quellen Netz prüfen. In zu großen IPv4-Netzen bildet `GroupByPrefix` Teilnetze je /16 mit mindestens 3 Quellen (Hoster 2). `BuildBlockList` erzeugt die Sperrliste in drei Modi; nicht öffentliche Adressen (`IsPublic`: ohne RFC 1918, Loopback, Link-Local, CGNAT, ULA) bleiben außen vor.

Abfrage (`CymruAsnService.LookupAsync`): `<reverse-ip>.origin.asn.cymru.com` bzw. `origin6.asn.cymru.com` liefern ASN, Präfix, Land und Registry, `AS<n>.asn.cymru.com` den Namen. Bis zu 12 Abfragen parallel, 3 s Zeitüberschreitung, ein Wiederholungsversuch; Ergebnisse (auch „nicht announciert“) bleiben für die Sitzung im Zwischenspeicher. Nur DNS, kein HTTP. Der Schalter `tglAsLookup` nutzt `ExternalLookups.SetNetworkInfo` (Datenschutz-Stufe „Netz- und Betreiberzuordnung“, wie im Traceroute) und ist standardmäßig aus; ohne ihn bleiben die Reiter Betreiber und Netze leer.

Oberfläche: Eingabefeld mit Einfügen, Datei-öffnen und Drag & Drop (`PreviewDragOver`/`PreviewDrop` am Dialog, damit das Textfeld die Datei nicht abfängt); Logs über 400 000 Zeichen bleiben im Speicher und das Feld zeigt nur eine Notiz. Reiter: Betreiber, Netze, IP-Adressen, Konten, Tageszeit, Sperrliste. Export: PDF (`ExportReport`), CSV (IP-Tabelle, Formelzeichen entschärft, Kunden-Nr-Spalte), TXT (reine Sperrliste ohne Kopfzeile).

© 2026 DATEXT

6. Navigationsgruppe: Server & Dienste

6.1 AD-Suche (ActiveDirectoryDialog)

Klasse: `ActiveDirectoryDialog` (Dialogs/ActiveDirectory/), Partials `.Query`, `.Model`, `.Export` **Technologie:** PowerShell mit dem ActiveDirectory-Modul (`Get-ADComputer`, `Get-ADUser`, `Get-ADDomainController`, `Get-ADDomain`, `Get-ADObject`)

Fragt Active Directory ab und zeigt Computer und Benutzer in zwei Tabellen. Die Suche kennt Platzhalter (*), die Modi „Enthält“ und „Beginnt mit“ sowie Filter (aktiviert/deaktiviert, nur gesperrte Konten, Anmeldung in den letzten 30 Tagen, inaktiv seit 30/90/180/365 Tagen, nie angemeldet). Der Domänencontroller wird automatisch erkannt und lässt sich überschreiben; für andere Domänen sind Benutzer und Kennwort eingebbar. „Verbindung testen“ prüft die Anmeldung vorab.

- **Kennzahlen:** Kacheln unter der Toolbar, per Klick als Filter nutzbar. Computer: Gesamt, Aktiv, Deaktiviert, Inaktiv > 180 Tage, Nie angemeldet. Benutzer: Gesamt, Gesperrt, Deaktiviert, Inaktiv > 180 Tage, Kennwort läuft nie ab, Kennwort abgelaufen bzw. Änderung bei Anmeldung. Das Badge zeigt „x von y“.
- **Spalte „Kennwort“** (Benutzer): „Läuft nie ab (Konto)“, „Kein Ablauf (Richtlinie)“ (Kennwortrichtlinie ohne Höchstalter), „Änderung bei Anmeldung“ (`pwdLastSet = 0`), „Abgelaufen“ oder „Läuft ab TT.MM.JJJJ“ (`msDS-UserPasswordExpiryTimeComputed`).
- **Detailbereich** (in der Höhe veränderbar): UPN, Kennwort zuletzt gesetzt, Erstellungsdatum, Beschreibung, vollständiger OU-Pfad, Gruppen (je Objekt zwischengespeichert, mit Ladeanzeige), „DN kopieren“ und „Vorgesetzter“ (springt zum Konto des Vorgesetzten). Werte sind markier- und kopierbar.
- **Exakte letzte Anmeldung:** Die Spalte „Letzte Anmeldung (AD)“ zeigt `lastLogonTimestamp`, das nur alle 9–14 Tage repliziert wird. „Exakt ermitteln“ (Detailbereich) bzw. „Letzte Anmeldung exakt ermitteln (alle DCs)“ (Kontextmenü) fragt `lastLogon` auf jedem DC der Domäne einzeln ab; vorher prüft der Dialog jeden DC mit 1,5 s Timeout auf ADWS (TCP 9389). Angezeigt wird der neueste Wert mit liefernden DC und dem Wert je DC; nicht abfragbare DCs werden genannt („Wert evtl. zu alt“). In der Liste steht danach „TT.MM.JJJJ HH:MM (exakt)“; Sortierung, Inaktivitätsfilter, Kacheln und Export verwenden den neueren der beiden Werte. Der Wert wird bewusst nur gezielt je Konto ermittelt, nicht für die ganze Liste.
- **Netzwerk-Abgleich:** Vergleicht AD-Computer mit den Ergebnissen des Geräte-Scans (empfängt `SetScanDevices()`): IP (AD) gegen IP (Scan), Erreichbarkeit; Aktionen Ping, Traceroute und Port-Scan je Computer.
- **Export** des aktiven Reiters (auch des Netzwerk-Abgleichs) als PDF, CSV (Semikolon, BOM), Excel, JSON (mit Kennwortdaten und bereits geladenen Gruppen) und TXT (lesbare Spalten); der Kopf nennt Domäne, DC, Suchbegriff, aktive Filter und Trefferzahl. „Kopieren“ meldet „Kopiert“ ohne Meldungsfenster.

6.2 Gruppenrichtlinien (GpoAnalysisDialog)

Klasse: `GpoAnalysisDialog` (Dialogs/ActiveDirectory/), Partials `.Parse`, `.Run`, `.Export` **Technologie:** `gpresult /X` (XML-Auswertung), `gpresult /H` (HTML), `gpupdate`

Zeigt die angewendeten Gruppenrichtlinien für Computer und Benutzer. Ohne Administratorrechte sind nur die Benutzer-GPOs verfügbar (Hinweis im Dialog, „Als Administrator neu starten“). Die Rohdaten werden erst beim Öffnen des Reiters „Rohdaten“ geladen.

- **Kennzahlen** (Kacheln, als Filter klickbar): GPOs, Angewendet, Nicht angewendet, Verknüpfung deaktiviert, Verwaiste Verknüpfung, Erweiterungsfehler (öffnet den Reiter „Erweiterungen“) und

Langsame Verbindung. Infozeilen für Computer und Benutzer mit Konto, OU, Standort und letzter Verarbeitung.

- **Reiter:** Benutzer-GPOs, Computer-GPOs (mit Status, verknüpfter OU, Verarbeitungsreihenfolge und bei nicht angewendeten GPOs dem Grund), Erweiterungen (Status jeder clientseitigen Erweiterung mit letzter Ausführung, Dauer und Fehlercode), Sicherheitsgruppen (die von gpresult für die Filterung berücksichtigten Gruppen mit Suche, dazu die durchsuchten Container mit „Vererbung blockiert"), Sicherheitseinstellungen, Protokollierung und Rohdaten.
- **Detailbereich je GPO:** alle Verknüpfungen mit Position, Verarbeitung, deaktiviert und erzwungen; Sicherheitsfilter und WMI-Filter; Version AD/SYSVOL (eine Abweichung weist auf ein Replikationsproblem hin); beteiligte Erweiterungen; „GUID kopieren".
- **Aktionen:** „gpupdate /force" mit anschließender Neuanalyse (Rückfragen zur Abmeldung werden mit „Nein" beantwortet), „HTML-Bericht" öffnet `gpresult /H` im Browser.
- **Export** aller Bereiche als PDF und TXT, außerdem „GPO-Liste als CSV" und „gpresult-XML speichern". Der Status erscheint als Badge mit Text (keine Emojis).

6.3 MS-SQL-Analyse (SqlAnalysisDialog)

Klasse: `SqlAnalysisDialog` (Dialogs/SqlAnalysis/) **Technologie:** `System.Data.SqlClient`, Dynamic Management Views (`sys.dm_*`)

Verbindungstest und Zustandsanalyse von SQL-Server-Instanzen: Instanzsuche oder manuelle Eingabe (`SERVER` bzw. `SERVER\INSTANZ`), Windows- oder SQL-Authentifizierung, wählbare Verschlüsselung (mit Hinweisen für ältere Server ≤ 2014 bei Zertifikatsfehlern). Registerkarten **Verbindung**, **Gesundheit** (u. a. Wait-Statistiken), **Aktivität**, **Performance** und **Konfiguration**; Auswertungen u. a. zu Speicherbelegung, Index-Nutzung und fehlenden Indizes.

Bedienung und Darstellung: Enter in Server, Port, Benutzer oder Kennwort verbindet. Nach erfolgreicher Anmeldung werden Server, Port, Anmeldeart, Benutzer, Verschlüsselung und Zertifikatsoption gemerkt, ohne Kennwort. In den SQL-Spalten sind Zeilenumbrüche und Leerraum zusammengezogen; der Volltext erscheint als Tooltip. Alle Tabellen haben das Kontextmenü „Markierte Zeilen kopieren" (mit Spaltenköpfen) und zeigen bei leerem Ergebnis „Keine Einträge". Wartezeiten sind lesbar (z. B. „3,3 min"). Bei der Express-Edition erscheint die tatsächliche Speichergrenze (1.410 MB) statt „Unbegrenzt", die Auslastung wird darauf bezogen. Die Code-Datei ist in Partial-Dateien aufgeteilt (`.Models`, `.Connection`, `.Health`, `.Activity`, `.Databases`, `.Analysis`, `.Procedures`, `.Maintenance`, `.Security`, `.LiveMonitor`, `.Export`).

6.4 IIS-Analyse (IisAnalysisDialog)

Klasse: `IisAnalysisDialog` (Dialogs/ServerServices/, mit `.Export.cs`) **Logik:** `IisConfigReader` (`Services\IisConfigAnalysis.cs`), `IisFtpReader` (`Services\IisFtpAnalysis.cs`), `IisRuntimeReader` (`Services\IisRuntimeState.cs`), `IisLogAnalyzer` (`Services\IisLogAnalysis.cs`), `FtpLogAnalyzer` (`Services\FtpLogAnalysis.cs`), gemeinsame Befundliste `ServerFinding` (`Services\ServerFindings.cs`) **Technologie:** XML (`XDocument`, DTD verboten), W3C-Protokolle, `X509Store`, `netsh http show sslcert`, Registry (Schannel), COM `Microsoft.ApplicationHost.AdminManager`, `ServiceController`, `CymruAsnService`

Die Konfiguration wird als XML gelesen (`applicationHost.config`, Standardpfad `%windir%\System32\inetsrv\config`); damit funktioniert auch eine kopierte Datei, ohne IIS-Verwaltungsmodul. Ausgelesen werden Pools (mit `applicationPoolDefaults`), Sites, Bindings, Anwendungen und die Abschnitte `system.webServer` global sowie je `<location>` (directoryBrowse, httpErrors, customHeaders, Authentifizierung). Lokal zusätzlich: Zertifikat-Zuordnung (HTTP.sys über `netsh`, sprachunabhängig geparst; Daten aus den Speichern „Eigene" und „WebHosting" des Computers), die `web.config` im Stammverzeichnis jeder Anwendung (Werte von Kennwörtern werden nie ausgegeben) und

ausdrücklich aktivierte Schannel-Protokolle. Die Prüfung steht in `Evaluate`, `CheckWebServerSection`, `CheckWebConfigs`, `CheckSchannel`.

Die Protokollauswertung liest W3C-Dateien zeilenweise (Spalten laut `#Fields:`, mehrere Dateien und Ordner rekursiv, Zeitfenster relativ zum letzten Eintrag; ältere Dateien werden übersprungen) und aggregiert Statuscodes mit Unterstatus, URLs (Anfragen, Fehler, mittlere und größte Zeit), Absender (404, 401, 5xx, Pfade, Sondierungstreffer) und Stunden. Eine Adresse gilt als Sondierung bei mindestens drei Treffern auf typische Angriffspfade (`Probe`-Muster) oder bei mindestens 15 und 60 % „404“, als Anmeldeversuch ab 10 und als Brute-Force ab 50 Antworten „401“. Die Betreiber-Abfrage (nur öffentliche Adressen, höchstens 500 je Lauf) ist wie in der Firewall-Log-Auswertung standardmäßig aus (`ExternalLookups.SetNetworkInfo`).

Laufzeitstatus (`IisRuntimeReader`, `Services\IisRuntimeState.cs`, nur bei der lokalen Konfiguration): Dienste W3SVC, WAS, FTPSVC über `ServiceController`; Status von Sites, `ftpServer` je Site und Anwendungspools über die COM-Schnittstelle `Microsoft.ApplicationHost.AdminManager` (spät gebunden mit `dynamic`, Abschnitte `system.applicationHost/sites` und `applicationPools`, Eigenschaft `state`: 0 Starting, 1 Started, 2 Stopping, 3 Stopped, 4 Unknown). Der Website-Status wird nur für Sites mit HTTP/HTTPS-Binding gesetzt, der FTP-Status nur für Sites mit FTP-Binding. Scheitert die Abfrage (keine Administratorrechte), entsteht ein Hinweisbefund (Bereich „Status“); ist der zuständige Dienst beendet, steht „Dienst beendet“. Die Bewertung steht in `IisRuntimeReader.Evaluate`.

FTP (`IisFtpReader`, `Services\IisFtpAnalysis.cs`): Sites mit `ftp`-Binding; Werte aus `site/ftpServer`, sonst `siteDefaults/ftpServer`, sonst Schema-Standard aus `FTP_schema.xml` (SSL-Richtlinien `SslRequire`, anonyme und Standard-Authentifizierung aus, `userIsolation` `None`, Protokollierung an). Regeln aus `system.ftpServer/security/authorization` global und in `<location path="Site">` bzw. `Site/Unterordner`; global außerdem `denyByFailure` und `firewallSupport` (passiver Portbereich); bei `centralLogFileMode="Central"` der zentrale Protokollordner. Das FTP-Zertifikat steht als `serverCertHash` in der Konfiguration und wird lokal in denselben Speichern gesucht wie die HTTPS-Zertifikate (`ResolveCertificates`).

FTP-Protokolle (`FtpLogAnalyzer`, `Services\FtpLogAnalysis.cs`): `IsFtpLog` erkennt eine Datei am Ordner `FTPSVC…` oder an `x-session`/`x-fullpath` in `#Fields:`; der Dialog teilt die Dateien danach auf (Web-Protokolle gehen unverändert an `IisLogAnalyzer`) und nimmt bei „Dieser Server“ abweichende FTP-Protokollordner aus der Konfiguration dazu. Sitzungen über `x-session` (sonst `c-ip:c-port`); Anmeldung = `PASS` mit 230 bzw. 530; TLS gilt nach `AUTH` mit 234 oder bei Serverport 990; Uploads `STOR`/`APPE`/`STOU` mit 226, Downloads `RETR` mit 226, Löschungen `DELE`/`RMD` mit 250. Auffälligkeiten je Adresse: Erfolg nach mindestens 10 Fehlversuchen, Brute-Force ab 50, Anmeldeversuche ab 10, Kontenraten ab 5 Fehlversuchen auf 5 Konten; Konten mit mindestens 20 Fehlversuchen gelten als angegriffen. Anonym sind `anonymous`, `ftp` und `…\IUSR`; deren Anmeldungen zählen nicht als „ohne TLS“.

Ausgabe: Reiter Befunde, Sites, Zertifikate, Anwendungspools, FTP, Statuscodes, Absender, Häufige/Langsame/Fehlerhafte URLs, Tageszeit, FTP-Absender, FTP-Konten; Export als PDF (`InventoryPdfExporter.ExportReport`), CSV (Befunde, Absender, FTP-Absender) und Zwischenablage. Offline geprüft gegen die Konfiguration und die Protokolle des Entwicklungsrechners (Status: eine Site, sechs Pools, Dienste), eine kopierte Konfiguration mit eingefügten FTP-Sites und nachgebaute FTP-Protokolle; nicht geprüft: eine echte FTP-Site und echte FTP-Logs, gestoppte Sites/Pools, Lauf ohne Administratorrechte.

6.5 Hyper-V-Analyse (HyperVDialog)

Klasse: `HyperVDialog` (Dialogs/ServerServices/, mit `.Export.cs`) **Logik:** `HyperVReader` (`Services\HyperVAnalysis.cs`), `VhdReader` (VHDX/VHD-Kopf), `ServerFinding` **Technologie:** WMI/`System.Management` (`root\virtualization\v2`, `root\cimv2`), `EventLogReader`

Der Host wird lokal (`.`) oder über `\\Host\root\virtualization\v2` (Identität des Aufrufers, `PacketPrivacy`) gelesen. VMs sind die `Msvm_ComputerSystem`-Objekte mit GUID als Namen (der Host trägt

den Rechnernamen, die Beschriftung ist lokalisiert und taugt nicht zur Unterscheidung). Je VM werden über die Zuordnungen `Msvm_SettingsDefineState` und `Msvm_VirtualSystemSettingDataComponent` Generation, Version, Prozessor, Arbeitsspeicher (dynamisch), Festplatten (`Msvm_StorageAllocationSettingData`), Netzwerkkarten und Switch-Verbindungen (`Msvm_EthernetPortAllocationSettingData`) gelesen; Prüfpunkte kommen aus `Msvm_VirtualSystemSettingData` mit Typ „Snapshot“ (`VirtualSystemIdentifier` = VM-Kennung); bei laufenden VMs Heartbeat (`Msvm_HeartbeatComponent`) und Gastdaten (`Msvm_KvpExchangeComponent`).

Größe, Art und Elternverweis der Festplatten stammen direkt aus dem Dateikopf: VHDX über Regionstabelle und Metadatenbereich (virtuelle Größe, Dateiparameter, Elternlokator), VHD über das Fußsegment `conectix`. Die Kette wird bis acht Ebenen verfolgt; ein fehlender übergeordneter Datenträger ist kritisch. Remote-Pfade werden über die Admin-Freigabe (`\\Host\D$\…`) gelesen. Die Laufwerke kommen aus `Win32_LogicalDisk`; „mögliches Wachstum“ ist die Summe aus maximaler Größe minus Dateigröße aller nicht festen Datenträger je Laufwerk. Fehler aus den Protokollen VMMS-Admin, Worker-Admin und Compute-Admin (14 Tage, nur lokal) werden nach ID gruppiert; ein Fehler dabei bricht die Auswertung nicht ab, er steht als Hinweis im Reiter „Host und Speicher“.

Geteilte Datenträger: `ReadVmDetails` liest `PersistentReservationsSupported` der `Msvm_StorageAllocationSettingData` (`HyperVDisk.Shared`); `ReadDisks` setzt `SharedWith` für Dateien, die mehrere VMs einbinden (Pfadvergleich ohne Groß-/Kleinschreibung), und zählt sie bei `VmUsed`/`Growth` nur einmal. `Evaluate` unterscheidet geteilt (Hinweis), mehrfach eingebunden ohne Merkmal (Warnung, kritisch bei mehr als einer laufenden VM) und geteilt auf `DriveKind == "Laufwerk"` (Warnung). VHD Set: `FillSet` (nur Tiefe 0, Endung `.vhds`) summiert `Name_*.avhdx` im Ordner, setzt `ChainDepth` auf die Teilezahl und nimmt `VirtualSize` aus dem Kopf des jüngsten Teils; das Setformat wird nicht geparst. Pass-Through: ResourceSubType „Physical Disk Drive“; `ReadPassThrough` liest die Nummer aus `\\.\PHYSICALDRIVEn` oder über `Msvm_DiskDrive.DriveNumber`, `ReadPhysicalSizes` Größe und Modell aus `Win32_DiskDrive`; diese Einträge laufen nicht durch `VolumeOf` (der Pfad `\\.\…` würde sonst als SMB-Freigabe gelten) und nicht durch den VHDX-Kopfparser.

Speicherorte bestimmt `HyperVReader.VolumeOf` nach Pfad: `X:\ClusterStorage\<Volume>` = Cluster Shared Volume, `\\Server\Freigabe` = SMB-Freigabe, sonst Laufwerksbuchstabe. Laufwerke kommen aus `Win32_LogicalDisk`, CSV und Freigaben über `GetDiskFreeSpaceEx` auf dem Pfad (Remote-Hosts: Admin-Freigabe `\\Host\C$\ClusterStorage\…`). `HyperVResult.IsCluster` ist gesetzt, wenn der Dienst `ClusSvc` existiert oder ein CSV vorkommt; dann erscheint ein Hinweisbefund (Bereich „Cluster“). Die Prüfung „Systemlaufwerk“ gilt nur noch für echte Laufwerksbuchstaben.

Die Bewertung steht in `HyperVReader.Evaluate` (Schwellen: freier Platz unter 15 % bzw. 5 % oder 10 GB, Prüfpunkte ab 7 bzw. 30 Tagen oder mehr als drei, vCPU-Verhältnis über 4:1, Arbeitsspeicher über 90 % des Hosts). Export als PDF, CSV (VMs, Befunde) und Zwischenablage. Offline geprüft gegen den Hyper-V-Host des Entwicklungsrechners (sechs ausgeschaltete VMs, Prüfpunkte, AVHDX-Ketten); nicht geprüft: Remote-Hosts, laufende VMs mit Integrationsdiensten (Heartbeat, Gastdaten), Replikation, ein echter Cluster (Pfaderkennung und freier Platz sind offline getestet) und freigegebene Datenträger (Shared VHDX).

6.6 AD-Sicherheit & Zustand (AdHealthDialog)

Klasse: `AdHealthDialog` (Dialogs/ActiveDirectory/, mit `.Export.cs`) **Logik:** `AdHealthReader` (`Services\AdHealthAnalysis.cs`, `.Domain.cs`, `.Events.cs`), `AdLdap` (LDAP-Hilfe), `ServerFinding` **Technologie:** `System.DirectoryServices.Protocols` (`LdapConnection`, Negotiate mit Signierung und Verschlüsselung, seitenweise Suche), `DnsClient`, SNTP über UDP 123, `RegistryKey.OpenRemoteBaseKey`, `EventLogReader`/`EventLogSession`

Die Analyse läuft ohne RSAT und ohne PowerShell (anders als `ActiveDirectoryDialog`) und ist rein lesend. Ohne Server und Benutzer verbindet sie sich mit der DNS-Domäne des Rechners über die aktuelle Anmeldung; mit Server, Benutzer und Kennwort geht sie gegen eine fremde Domäne (`NetworkCredential`, `DOMÄNE\Benutzer` wird aufgeteilt). Das Kennwort wird nicht gespeichert.

- **Konten** (`AnalyzeAccounts`): privilegierte Gruppen über die SID (`-512` Domänen-Admins, `-519`/`-518` nur in der Gesamtstruktur-Stammdomäne, `S-1-5-32-544/548/549/550/551`, `-520`), Mitglieder mit `memberOf:1.2.840.113556.1.4.1941:=` (verschachtelt). Benutzer und Computer werden mit `userAccountControl`, `lastLogonTimestamp`, `pwdLastSet`, `servicePrincipalName`, `adminCount` und LAPS-Ablaufattributen gelesen; Exchange-Systemkonten (`HealthMailbox…`, `SystemMailbox{…}`) werden ausgenommen. LAPS gilt als vorhanden, wenn das Schema `ms-Mcs-AdmPwd` oder `msLAPS-Password` kennt; die Abdeckung zählt Computer mit Ablaufattribut.
- **Domäne** (`AnalyzeDomain`): Funktionsebenen aus der RootDSE, Schema `objectVersion`, Richtlinie am Domänenkopf, PSO unter `CN=Password Settings Container`, FSMO über `fSMORoleOwner` an Schema, Partitions, Domäne, RID Manager$ und Infrastructure, Vertrauensstellungen über `trustAttributes` (SID-Filterung: Bit 4 oder Gesamtstruktur-Vertrauen ohne Bit 64), Papierkorb über `msDS-EnabledFeatureBL`, Standorte und Subnetze aus der Konfiguration.
- **Domänencontroller** (`AnalyzeDcs`): Ports per TCP-Connect, SRV-Einträge `_ldap._tcp.dc._msdcs`/`_kerberos._tcp.dc._msdcs`, SNTP-Zeitabweichung (über 2 min Warnung, über 5 min kritisch), SYSVOL aus `msDFSR-Flags` an `CN=DFSR-GlobalSettings` (48 = „Eliminated“, fehlt das Objekt, gilt FRS). Der Replikationsstatus kommt je Controller aus `msDS-NCReplInboundNeighbors` an den Namenskontext-Köpfen (Domäne, Konfiguration, Schema, Anwendungspartitionen); der Wert ist ein XML-Text mit angehängtem Nullzeichen. LDAP-Signierung (`LDAPServerIntegrity`), Channel Binding (`LdapEnforceChannelBinding`), `SMB1`, `LmCompatibilityLevel` und `SignSecureChannel` liest die Remote-Registrierung, nur mit der aktuellen Anmeldung und mit 8 s Zeitgrenze.
- **Ereignisse** (`AnalyzeEvents`): XPath-Abfrage auf das Sicherheitsprotokoll jedes Controllers (4740, 4625, 4771, 4776 nur Fehler), neueste zuerst, höchstens 20 000 Ereignisse je Controller; Auswertung über das XML (`EventData/Data[@Name]`). Quellen werden nach Adresse (sonst Arbeitsstation) zusammengefasst; Schwellen: ab 10 Konten und 20 Versuchen „Password Spraying“, ab 100 Versuchen an höchstens 3 Konten „Brute-Force“, öffentliche Adresse (`FirewallLogAnalysis.IsPublic`) ab 5 Versuchen kritisch.

Ausgabe: Reiter Befunde, Konten, Privilegierte, Domänencontroller, Replikation, Domäne, Sperrungen, Fehlanmeldungen mit Filterfeld; Export als PDF (`InventoryPdfExporter.ExportReport`) und CSV (Befunde, Konten). Teilprüfungen, die scheitern (Rechte, Erreichbarkeit), werden als Hinweis vermerkt und brechen die Analyse nicht ab.

6.7 DNS-Server-Analyse (DnsServerDialog)

Klasse: `DnsServerDialog` (Dialogs/ServerServices/, mit `.Export.cs`) **Logik:** `DnsServerReader` (`Services\DnsServerAnalysis.cs`), Hilfen `ServerWmi` und `ServerProbe` (`Services\ServerFindings.cs`) **Technologie:** WMI `root\MicrosoftDNS` (`MicrosoftDNS_Server`, `_Zone`, `_ResourceRecord`, `_Statistic`), DnsClient für die Prüfung von außen, `EventLogSession` für das Protokoll „DNS Server“.

Ablauf in `Read`: Konfiguration, Prüfung von außen, Ereignisse, Bewertung. Scheitert WMI an den Rechten, bleibt `ConfigRead` falsch und `ConfigProblem` nennt den Grund; die Prüfung von außen läuft dann mit der Domäne dieses Rechners statt mit den Zonen. Einträge liest `ReadRecords` je Zone als Vorwärts-Aufzählung (höchstens 100 000 je Zone, 5 000 Zeilen in der Liste). `Timestamp` zählt Stunden seit 1601 (0 = statisch); veraltet ist ein Eintrag, der älter ist als die Summe der Alterungsintervalle der Zone, ohne Angabe 14 Tage (`IsStale`).

Die Regeln stehen in `Evaluate` und arbeiten nur auf `DnsServerResult`. Bewusst nicht enthalten: ein Versuch eines Zonentransfers (AXFR) von außen, weil er auf fremden Servern als Angriff gewertet werden kann; die Übertragungsregel kommt allein aus der Konfiguration.

6.8 DHCP-Server-Analyse (DhcpServerDialog)

Klasse: `DhcpServerDialog` (Dialogs/ServerServices/, mit `.Export.cs`) **Logik:** `DhcpServerReader` (`Services\DhcpServerAnalysis.cs`), `DhcpNative` (`Services\DhcpServerNative.cs`), `AdLdap` für die

Autorisierung **Technologie:** `dhcpsapi.dll` per P/Invoke (RPC, ohne RSAT), LDAP, Dateizugriff auf das Überwachungsprotokoll.

`DhcpNative` liest die verschachtelten Antworten über feste Abstände für x64; die Anwendung gibt es nur für win-x64. Die Abstände stehen als Kommentar an jeder Stelle und wurden per C-Programm gegen `dhcpsapi.h` geprüft. `DhcpRpcFreeMemory` gibt nur den übergebenen Block frei, innere Blöcke bleiben bis zum Prozessende. Das Feld `SharedSecret` der Failover-Beziehung wird nie gelesen. Rückgabecodes: 5 = Dienst antwortet ohne Rechte, 1753 oder 1722 = kein Dienst bzw. kein RPC erreichbar.

Autorisierung: `ReadAuthorized` liest `dHCPClass`-Objekte unter `CN=NetServices,CN=Services` der Konfiguration und ruft bei jedem `DhcpGetMibInfo` auf. Überwachungsprotokoll: `ParseAudit` wertet `DhcpSrvLog-*.log` nach Ereigniskennung aus, Kopfzeilen und kaputte Zeilen werden übersprungen. Für andere Rechner wird der Ordner über die Administratorfreigabe gelesen (`ToAdminShare`). Die Schwellen `UsageWarn` und `UsageCrit` stehen als Konstanten in `DhcpServerReader`.

6.9 Terminalserver-Analyse (RdsDialog)

Klasse: `RdsDialog` (Dialogs/ServerServices/, mit `.Export.cs`) **Logik:** `RdsReader` und `WtsNative` (`Services\RdsAnalysis.cs`) **Technologie:** eigene RDP-Aushandlung über TCP, `SslStream`, WMI `root\cimv2\TerminalServices`, `StdRegProv`, `wtsapi32.dll`.

Der Verbindungsaufbau (`RunProbes`) sendet einen X.224 Connection Request mit `RDP_NEG_REQ` und wertet `RDP_NEG_RSP` bzw. `RDP_NEG_FAILURE` aus. Drei Anfragen: TLS und CredSSP (danach TLS-Handschlag für Zertifikat und Version, dann Abbruch), nur TLS (Code 5 = NLA erzwungen, `InterpretTlsOnly`), kein TLS (Code 1 oder 5 = abgelehnt, `InterpretLegacy`). Andere Codes gelten als nicht eindeutig und erzeugen keinen Befund. Anmeldedaten werden nie gesendet.

Registrierungswerte kommen über `StdRegProv`, damit der Dienst „Remoteregistrierung“ nicht nötig ist; der Richtlinienschlüssel `Policies\Microsoft\Windows NT\Terminal Services` geht vor `WinStations\RDP-Tcp`. `LicensingType`: 1 = Verwaltung, 2 = Pro Gerät, 4 = Pro Benutzer, 5 = nicht konfiguriert. Lizenzregeln und Zeitlimits gelten nur bei `TerminalServerMode` = 1 (`EvaluateSessionHost`). Sitzungen ohne Benutzername (Dienste, Listener) werden ausgelassen; `WTSEnumerateSessions` meldet ohne Rechte auf entfernten Servern 5 oder 259. Die Schwellen stehen als Konstanten in `RdsReader`.

Aktionen (`RdsDialog.Actions.cs`): `RdsReader.LogoffSession` ist die einzige ändernde Funktion aller Server-Analysen. Sie liest die Sitzungen zuerst neu und bricht ab, wenn `IsSameSessionAs` (Nummer, Benutzer, Anmeldezeit) nicht mehr zutrifft, weil Windows Sitzungsnummern neu vergibt. `WTSLogoffSession` läuft ohne Warten; `RefreshSessions` fragt danach bis zu 15 Sekunden nach, ob die Sitzung verschwunden ist, und bewertet neu. Ein Rechtsklick außerhalb der Auswahl setzt die Auswahl auf diese Zeile. `BtnLicenseManager_Click` startet `licmgr.exe` aus dem Systemordner.

Auslastung, Profile, Anmeldungen (`Services\RdsAnalysis.Usage.cs`): `WtsProcessNative` liest `WTSEnumerateProcessesEx` auf Stufe 1 und übernimmt bewusst nur Zahlen, keine Prozessnamen. `AggregateLoad` summiert zwei Messungen im Abstand von 3 Sekunden je Sitzung; die CPU ist eine Momentaufnahme in Prozent des gesamten Servers, die Speichersumme zählt gemeinsam genutzten Speicher mehrfach. Profile kommen aus `Win32_UserProfile` (ohne Systemprofile), die Größe aus `MeasureFolder` lokal oder über die Administratorfreigabe, höchstens 60 Sekunden je Analyse und ohne Verknüpfungspunkte; nicht lesbare Ordner fehlen in der Summe. `VHDLocations` unter `SOFTWARE\FSLogix\Profiles` zeigt, dass die lokale Größe nicht den Platzbedarf beschreibt.

Fehlversuche stammen aus 4625 im Sicherheitsprotokoll (nur `LogonType` 3 und 10, also kein Entsperren) und 140 in `RdpCoreTS/Operational`; höchstens 20 000 Ereignisse je Protokoll, 14 Tage. Typ 3 kann auch von Dateifreigaben stammen und zählt deshalb als „Netzwerk“: `ClassifyFailures` stuft das aus dem eigenen Netz nur als Hinweis ein. Schwellen: öffentliche Adresse ab 5 Versuchen kritisch, ab 10 Konten und 20 Versuchen „Password Spraying“, ab 100 Versuchen „Brute-Force“. Erfolgreiche Anmeldungen kommen aus 21 und 25 in `LocalSessionManager/Operational`, nicht aus 1149, weil dort der Benutzername fehlen kann.

© 2026 DATEXT

`IsAfterFailures` meldet einen Erfolg, wenn dieselbe Quelle vorher mindestens auf Warnstufe gescheitert ist.

7. Navigationsgruppe: Packet Capture

7.1 PKTMON Capture (PktmonCaptureDialog)

Klasse: `PktmonCaptureDialog` (Dialogs/SystemInfo/) **Technologie:** `pktmon.exe`, `Etl2PcapngService`; gemeinsame Bausteine `TraceFileList`, `TraceTools`, `ICaptureSession`

Steuert Windows Packet Monitor (PKTMON) für die Paketaufzeichnung. Voraussetzung: Windows 10 Build 1809+ und Administratorrechte (Neustart als Administrator über `App.RestartAsAdministrator()`). Der Dialog ist wie der E-Mail-Check aufgebaut: Einstellungen links, Protokoll rechts, verschiebbare Trennlinie (Verhältnis gespeichert, Doppelklick setzt 42/58 zurück), in schmalen Fenstern Umschalter „Einstellungen | Protokoll“.

- **Optionen** (werden gespeichert): Komponenten (alle / nur Netzwerkkarten, `--comp nics`), Pakete (alle / nur verworfene, `--type drop`), Paketgröße (`--pkt-size`), maximale Dateigröße in MB (`-s`) und Protokollmodus (Ringpuffer, mehrere Dateien, Speicher).
- **Live-Kacheln:** Laufzeit, Pakete, verworfene Pakete und Dateigröße aus `pktmon counters --json`, alle 2 s.
- **Filter:** Eingabe mit Prüfung, Vorlagen (DNS, HTTPS, SMB, RDP, DHCP, Ping) und Anzeige der aktiven Filter aus `pktmon filter list`. Filter gelten erst für die nächste Aufzeichnung (Hinweis im Dialog).
- **Aufzeichnungen:** die letzten 15 ETL-/PCAPNG-Dateien aus dem neuen und dem alten Ordner mit „PCAPNG“ (umwandeln), „Wireshark“ (gefunden über „App Paths“ in der Registry) und „Im Ordner zeigen“. Nach dem Stoppen wird automatisch nach PCAPNG umgewandelt (abschaltbar).
- **Abfragen:** „Paketverluste“ (`counters --type drop -r`, mit letztem Grund je Komponente), „Zähler“ und „Komponenten“ schreiben ins Protokoll.
- **Speicherort:** `%LocalAppData%\DATEXT\NetworkTraces` (früher „Dokumente\DATEXT-Diagnostics\NetworkTraces“; bei OneDrive-Umleitung wurden ETL-Dateien bis 512 MB mitsynchronisiert). Vorhandene Dateien im alten Ordner zeigt der Dialog weiter an.
- **Protokoll und Export:** farbiges Protokoll ([OK]/[!]/[X]/[i]) mit Zeilenumbruch-Schalter; Kopieren und Export als TXT oder PDF enthalten Rechner, Speicherort, Zähler und Filter.

7.2 NETSH Trace (NetshTraceDialog)

Klasse: `NetshTraceDialog` (Dialogs/SystemInfo/) **Technologie:** `netsh trace`, `Etl2PcapngService`; gemeinsame Bausteine wie PKTMON

Steuert NETSH Trace für umfassende Windows-Netzwerkdiagnose, im gleichen Aufbau wie PKTMON (Einstellungen links, Protokoll rechts, verschiebbare Trennlinie). Erfordert Administratorrechte.

- **Szenarien:** Liste mit Mehrfachauswahl und Beschreibung (WLAN, LAN, Internet u. a.), die Auswahl wird gespeichert. „Anbieter“ zeigt die ETW-Anbieter der gewählten Szenarien statt der kompletten Anbieterliste.
- **Paketmitschnitt:** an/aus, Schnittstellen (physisch / virtueller Switch / beides), Filter nach IP-Adressen und Protokoll (geprüft; netsh kennt keine Portfilter).
- **Datei:** maximale Größe, „Wenn voll“ (überschreiben / anhalten), Zusatzbericht (.cab) abschaltbar (`report=disabled`, Stoppen deutlich schneller), `persistent=yes` für Aufzeichnungen über einen Neustart.
- **Live-Kacheln:** Laufzeit, Dateigröße, Szenarien, Paketmitschnitt.
- **Aufzeichnungen:** ETL, CAB, PCAPNG und Berichte mit „PCAPNG“, „Bericht“, „Wireshark“, „Öffnen“ und „Im Ordner zeigen“; Konvertierung der ETL-Ausgabe nach PCAPNG über `Etl2PcapngService` (Tool in `Tools\etl2pcapng\`, Ausgabe `.pcapng`).

© 2026 DATEXT

- **Protokoll und Export:** farbiges Protokoll, Statuszeile statt Meldungsfenster, Export als TXT oder PDF. Speicherort wie bei PKTMON.

© 2026 DATEXT

8. Navigationsgruppe: Speedtests

8.1 Internet-Speedtest (InternetSpeedtestDialog)

Klasse: `InternetSpeedtestDialog` (Dialogs/Performance/) **Technologie:** Ookla Speedtest CLI, JSON-Ausgabe

Führt einen professionellen Internet-Geschwindigkeitstest durch:

- Nutzt das offizielle Ookla Speedtest CLI-Tool
- Tool wird bei Erstverwendung automatisch heruntergeladen und im AppData-Verzeichnis gespeichert
- Live-Messung im jsonl-Format mit Zwischenständen alle 250 ms (Phase und Fortschritt im Kartenkopf); Serverauswahl (nächstgelegene Server oder „Automatisch“, wird gemerkt)
- Ergebnisse: Download, Upload, Ping, Jitter, Paketverlust, Testserver, Latenz unter Last, Ergebnislink; Verbindungsklasse mit Eignung, Transferzeiten und eigene Berechnung
- Messverlauf als `ObservableCollection<MessVerlaufEintrag>` mit Durchschnittswertzeile
- JSON-Parsing der Speedtest-CLI-Ausgabe via `Newtonsoft.Json`

8.2 Netzwerk-Speedtest (NetworkBandwidthDialog)

Klasse: `NetworkBandwidthDialog` (Dialogs/Performance/) **Technologie:** `System.Net.NetworkInformation`, Datei-I/O, `NetworkInterface`-Statistiken

Misst die LAN-Bandbreite durch Dateitransfer zu einem freigegebenen Netzwerkpfad:

- Der Adapter wird automatisch ermittelt (der Weg, über den Windows den Server laut Routing-Tabelle erreicht); angezeigt werden Linkgeschwindigkeit und bei echten Adaptern die gemessene Auslastung, virtuelle Adapter sind gekennzeichnet
- Latenzmessung, mehrere Läufe mit Min/Max, Bewertung relativ zur Linkgeschwindigkeit, zuletzt genutzte Freigaben
- Puffergröße: 1 MB (`BUFFER_SIZE`)
- Messung via tatsächlichem Dateitransfer (Schreiben/Lesen)
- Session-Verlauf `ObservableCollection<LanHistoryEntry>`
- Pfad-Validierungscheck mit `DispatcherTimer` (Debounce)

8.3 Disk-Speedtest (DiskCheckDialog)

Klasse: `DiskCheckDialog` (Dialogs/Performance/) **Technologie:** DiskSpd (Microsoft), WMI für VM-Erkennung

Misst die Festplatten-Leistung mit dem Microsoft DiskSpd-Tool:

- DiskSpd wird bei Erstverwendung automatisch heruntergeladen
- VM-Erkennung via WMI (angepasste Schwellwerte für virtuelle Maschinen)
- Vier Teilmessungen nach dem Vorbild von CrystalDiskMark, ohne Windows-Cache (`-Su`) und mit Latenz (`-L`): sequenziell schreiben und lesen sowie 4K zufällig lesen mit QD32 und QD1
- Testdatei wählbar (256 MB / 1 GB / 4 GB, Standard 1 GB); Laufwerkstyp, Modell und Anschluss je Buchstabe (HDD, SATA-SSD, NVMe, USB, virtuell); Prüfung des freien Platzes; liegengebliebene Testdateien werden erkannt und lassen sich löschen
- Bewertung bezogen auf den Laufwerkstyp (z. B. „NVMe erreicht nur SATA-Niveau“) mit Begründung zur Eignung; Fortschritt mit Phase und Restzeit; Messverlauf mit CSV-Export
- Messverlauf `ObservableCollection<DiskHistoryEntry>`

© 2026 DATEXT

9. Navigationsgruppe: E-Mail & Domain

Alle drei Dialoge liegen unter `Dialogs/Mail/` und sind `UserControl`s mit demselben Aufbau: Bewertung (Statuszeilen mit Zähler, Probleme zuerst, Klick springt zur Stelle im Protokoll) und Verlauf links, farbiges Protokoll (`MailLogFormat`) rechts, verschiebbare Trennlinie, Meldungen in der Statuszeile. **Gemeinsame DNS-Auswertung:** SPF, DKIM, DMARC, MX, MTA-STS, TLS-RPT, DANE und BIMI kommen für alle drei aus dem Dienst `MailDomainAnalyzer`.

9.1 E-Mail-Check (EmailCheckDialog)

Klasse: `EmailCheckDialog` (Dialogs/Mail/, Partial-Dateien `.Tests`, `.Analysis`, `.Export`, `.Kim`, `.Layout`)
Technologie: MailKit, MimeKit, TLS/SSL, X.509-Zertifikate, MSAL und Microsoft Graph

Prüft den Mailserver zu einer E-Mail-Adresse:

- **Verbindungskarte** mit vier Statuszeilen: Verbindung, Verschlüsselung (TLS-Version, Cipher, Schlüsselaustausch), Zertifikat (gültig bis, Aussteller, „Anzeigen“) und Anmeldung (angebotene AUTH-Verfahren; Warnung bei AUTH ohne Verschlüsselung)
- **Server ermitteln** (Provider, SMTP-Server oder MX-Eintrag); **Alle Ports** prüft 587 (STARTTLS), 465 (TLS) und 25 gleichzeitig, mit Hinweisen wie „Port 25 oft vom Provider gesperrt“
- **Testmail** per SMTP (Erfolg, Relay-Test, MX-Direkttest und IMAP-Empfang in der Statuszeile). Bei **Microsoft 365** gibt es den Modus „Microsoft 365 (Graph)“: Anmeldung per OAuth2 im Browser (MSAL, öffentlicher Client ohne Secret, MFA möglich) und Versand über `POST /me/sendMail` (Berechtigungen `User.Read`, `Mail.Send`, `offline_access`), weil SMTP AUTH bei Exchange Online meist abgeschaltet ist. Der Modus wird nach „Server ermitteln“ vorgeschlagen. Getestet werden Anmeldung, Postfach und Versandrecht, nicht der SMTP-Einlieferungsweg. Die Graph-Testmail trägt dieselbe Signatur wie die SMTP-Testmail (`GraphMailTestService.Disclaimer`).
- **Sicherheitsanalyse** als Statuszeilen: SPF, DKIM, DMARC, MX, DNS-Sicht (interne Sicht bei privater MX-Adresse), MTA-STS, TLS-RPT, DANE, BIMI aus `MailDomainAnalyzer`; dazu die Prüfungen von diesem Rechner aus: MX auf Port 25, PTR und Blacklist der eigenen öffentlichen IP, TLS-Bewertung nach BSI TR-02102, Postfachzugriff (IMAP/POP3). Die SPF-Zeile nennt, ob die eigene öffentliche IP zugelassen ist. Die Reputation der Versand- und MX-IPs gehört bewusst zur E-Mail Domain-Analyse.
- **Protokolle** je Test in Reitern („#3 Analyse“, Kontextmenü, Überlauf-Menü), farbig mit `[OK]`/`[!]`/`[X]`/`[i]`; Export als PDF oder TXT
- **KIM/TI** (`.telematik`-Adressen): Konnektor-Suche, TI-DNS und Routing-Prüfung (TI-Netze 100.102.0.0/15 und 10.30.0.0/15) sind als Basis vorhanden, die Oberfläche ist ausgeblendet.

9.2 E-Mail Header-Analyse (EmailHeaderAnalysisDialog)

Klasse: `EmailHeaderAnalysisDialog` (Dialogs/Mail/, Partial-Dateien `.Parse`, `.Assessment`, `.Export`)
Technologie: Regex, Header-Parsing, `MailDomainAnalyzer`

Analysiert E-Mail-Header auf Authentifizierung, Routing und Verzögerungen:

- Bewertung mit Statuszeilen und Zähler, Probleme zuerst
- SPF-/DKIM-/DMARC-/BIMI-Ergebnisse aus `Authentication-Results`; die SPF-Gegenprüfung der Absender-IP nutzt `MailDomainAnalyzer` (`MailDomainParts.Spf` mit `senderIp`)
- Zustellweg aus den Received-Headern: IP und Protokoll/TLS je Station (`ESMTPS`, `version=TLS1_2`), Zeiten in Ortszeit, Verzögerung farbig, langsamste Station fett; Klick zeigt den vollständigen Header
- Spam-Score (X-Spam-Score)
- Online-Prüfungen (abschaltbar, wird gemerkt)

9.3 E-Mail Domain-Analyse (EmailAnalysisDialog)

Klasse: `EmailAnalysisDialog` (Dialogs/Mail/, Partial-Dateien `.Checks`, `.Export`; früher `DomainAnalysisDialog`) **Technologie:** `MailDomainAnalyzer`, DNS, HTTPS

Analysiert eine Mail-Domain:

- **SPF** nach RFC 7208 mit Zähler der DNS-Abfragen, Qualifier von `all`, `ptr`, zu weiten Netzen und erkannten Versanddiensten; **DKIM** über eine Selektorliste mit Schlüssellänge (leeres `p=` = widerrufen); **DMARC** mit `p`, `sp`, `pct`, `rua` und Vererbung von der Organisationsdomain
- **MTA-STS** (TXT und Richtlinie per HTTPS), **TLS-RPT**, **DANE/TLSA** für die ersten zwei MX, **Null-MX** (RFC 7505), **BIMI**
- **Reputation:** Versand-IPs aus SPF (`ip4` bis /29, `a`, `mx`) und MX-IPs gegen Blacklists, Domain gegen Spamhaus DBL und SURBL (über das System-DNS; ist Spamhaus darüber nicht erreichbar, steht ein Hinweis statt „sauber“)
- **DNS-Sicht:** Liefert der DNS-Server für den MX eine private Adresse (Split-DNS), weist eine Zeile darauf hin; fehlende Einträge gelten dann als Warnung „Fehlt (intern)“
- **Mail-Fluss:** bewertet Absender → Empfänger (Empfänger-MX bzw. Null-MX, strenge Anbieter wie Microsoft 365 und Google, SPF, DKIM, DMARC, Reputation)
- Export als Bericht mit Bewertung, Mail-Fluss und Protokoll

Der Dienst `DomainAnalysisService` ist nur noch für KIM-Domains zuständig (Anbieter-Erkennung, optionale TI-Analyse); die Standardprüfungen sind im `MailDomainAnalyzer` aufgegangen.

© 2026 DATEXT

10. Navigationsgruppe: Inventar

10.1 Lokale Inventarisierung (LocalInventoryDialog)

Klasse: `LocalInventoryDialog` (Dialogs/Inventory/) **Technologie:** WMI (u. a. `Win32_ComputerSystem`, `Win32_OperatingSystem`), Registry

Hardware- und Software-Erfassung des lokalen Rechners ohne Agent. Ohne Administratorrechte eingeschränkte Auswertung (schmale Hinweisleiste im Dialog).

- **Gemeinsame Erfassung:** Der Collector liegt einmal in `DATEXT.Shared\Inventory\InventoryCollector.cs` und wird von Diagnostics und Agent genutzt. Die Bereiche laufen parallel (typisch 19–22 s), die Statuszeile zeigt Schritt, laufende Bereiche und verstrichene Zeit. Windows-Features kommen über WMI, PowerShell/DISM/Registry bleiben Fallback.
- **Erfasst u. a.:** Patch-Stand (UBR), Zustand und Anschluss der Datenträger, Benutzer-Installationen aus geladenen Profilen, Autostart inkl. `WOW6432Node`. Sicherheitsprüfungen (NTLM-Stufe, SMB-Signierung, Credential Guard, Überwachungsrichtlinie, Windows LAPS auch über Intune) laufen auch ohne Domäne; Domänenteile (DC, GPOs, Kerberos) nur bei Mitgliedern.
- **Freigaben und Zugriffsrechte:** `CollectSharedFolders` liest die Freigaben (`Win32_Share`) und je Datenträger-Freigabe die Freigabeberechtigungen (`Win32_LogicalShareSecuritySetting.GetSecurityDescriptor`, nur mit Administratorrechten) und die NTFS-Berechtigungen (`DirectoryInfo.GetAccessControl`). `AccessEntry` hält Konto, SID, Erlauben/Verweigern, Rechtetext (Vollzugriff, Ändern, Lesen …), Schreibzugriff und Vererbung; `ShareAclStatus` und `NtfsAclStatus` unterscheiden „nicht erfasst“ (älterer Agent) von „ok“ und Fehlergründen. Kontonamen werden per SID aufgelöst (Zeitbudget 15 s, danach SID). Die Bewertung `BroadAccessWarning` verwendet die Schnittmenge aus Freigabe und NTFS. Ab Agent 1.1.3.
- **Drucker:** `CollectPrinters` liest zusätzlich `Shared` und `ShareName` (`Win32_Printer`) und je Drucker die Berechtigungen über die Druck-API (`OpenPrinter` mit READ_CONTROL, `GetPrinter` Stufe 3, Sicherheitsbeschreibung in `RawSecurityDescriptor`). Einträge je Konto werden zusammengefasst; Rechte: Drucken (0x8), Drucker verwalten (0x4), Dokumente verwalten (0x10). Umgeleitete Remote-Sitzungs-Drucker werden am Namen erkannt und nicht gelesen. `PrinterEntry.BroadAccessWarning` meldet breite Verwaltungsrechte. Ab Agent 1.1.4.
- **Gruppenrichtlinien:** Die angewendeten Computer-GPOs kommen zuerst aus dem lokalen Cache (`…\Group Policy\History`), sofort und ohne Netz. Danach wird der Weg zum Domänencontroller geprüft (DNS, TCP 389, Schnittstelle/Subnetz): lokal → frische Daten per `gpresult` samt Sicherheitsgruppen; nur über VPN → lokale Daten sofort mit Hinweis „Abfrage beim DC läuft“, `gpresult` läuft bis zu 180 s im Hintergrund und aktualisiert die Anzeige. Felder `gpoSource` und `dcPath`.
- **Ansicht:** Ergebnis direkt auf der Seite (gemeinsame `InventoryDetailView`, auch im Detailfenster der Agent-Inventare). Die lokale Inventarisierung ist zugleich die zweite Ansicht „Inventar & Bewertung“ der Übersicht; Exporte der Übersicht enthalten dann einen gemeinsamen **Systembericht**.
- **Reiter der Ansicht** (`InventoryDetailView`): Übersicht (mit Bewertung, Probleme zuerst; ein Klick öffnet den Bereich), System, Hardware, Software, Updates, Dienste, Netzwerk, Features, Benutzer, Gruppen, Autostart, BitLocker, Freigaben, Drucker, Sicherheit, Vergleich (Filter Alle/Neu/Entfernt/Geändert) und Protokoll. Erfasst werden die Bereiche Hardware, Software, Windows-Updates, Dienste, Netzwerkadapter, Autostart, Freigaben, Drucker, WSUS sowie Benutzer & Gruppen.
- **Speichern:** nur manuell. Standard „Inventar speichern (verschlüsselt)“ im Agent-Format (AES-256-GCM, nur mit Diagnostics lesbar), optional „Inventar als JSON (unverschlüsselt, für Import)“.

- **Vergleichen …:** lädt ein früheres Inventar (Datei oder Agent-Datei) und zeigt Unterschiede zu Software, Updates, Diensten, Benutzern, Gruppen, Autostart, Features, Freigaben, Druckern, BitLocker, Sicherheitsbefunden, Hardware und Systemwerten, filterbar nach Art und Bereich.
- **PDF:** Bericht mit der Bewertung direkt nach der Titelseite (`InventoryPdfExporter`).

10.2 Inventory Agent (InventoryAgentDialog)

Klasse: `InventoryAgentDialog` (Dialogs/Inventory/), Agent-Projekt `DATEXTAgent`

Hardware- und Software-Inventarisierung im Netzwerk über den DATEXT Inventory Agent (aktuell **Version 1.1.4**, signiert und in Diagnostics eingebettet; sie liefert Zugriffsrechte der Freigaben (ab 1.1.3) sowie Freigabe und Berechtigungen der Drucker; Inventare älterer Versionen bleiben lesbar). Der Agent richtet URL-Reservierung und Firewall-Regel beim Start selbst ein; ohne API-Key sind nur lesende Abfragen erlaubt, `/status` nennt Ablaufdatum, Key-Pflicht und letzte Erfassung. Das Installationsskript ist idempotent, prüft Administratorrechte und .NET 8 und liegt mit Deinstallationsskript und LIESMICH im Paket. Bei VPN wartet der Agent wie bei lokalem DC bis zu 90 s auf die Gruppenrichtlinien und liefert sonst die lokalen Daten. Der Dialog ist in `.Deploy`, `.Scripts`, `.Agents` und `.Import` geteilt. Registerkarten **Deployment** (Kunden-ID, Ablageort, Erfassungsmodus und -zeit, optionaler API-Key, HTTP-Port für den Pull-Modus, Laufzeit 1–30 Tage, Bereitstellung u. a. in einer Freigabe), **Agents im Netzwerk** (Status, Hostname, IP, Version, Modus, letzte Erfassung) und **Import & Auswertung**.

Erfassungsmodus und Sicherheit (Deployment): Der Agent läuft **Einmalig** (`RunOnce`: erfasst einmal, speichert die Datei und deinstalliert sich selbst, kein Netzwerkdienst und keine URL-Reservierung oder Firewall-Regel – das Ergebnis liegt nur als Datei im Ablageort), **Täglich** oder **Wöchentlich** (Wochentag und Uhrzeit hh:mm). Die Laufzeit beträgt 1–30 Tage; danach deinstalliert sich der Agent selbst. Der Kunde (Kunden-ID/Kundennummer, Pflichtfeld) steht in jeder Inventardatei. Der API-Key ist Pflicht (mindestens 16 Zeichen, erzeugbar, wird je Kunde gemerkt und per Windows-DPAPI geschützt gespeichert) und schützt Abruf, Soforterfassung und Deinstallation. Der Abruf läuft per HTTP im lokalen Netz (Standard-Port 7890); der Key schützt vor fremden Abrufen, die Übertragung selbst ist nur durch den in beiden Programmen hinterlegten Schlüssel verschlüsselt (`InventoryEncryption`, AES-256-GCM).

Bereitstellungswege: Dateien in einen (Netzwerk-)Ordner kopieren, Installationsskript anzeigen (Ausführung auf dem Zielrechner als Administrator), Startskript für eine Gruppenrichtlinie mit Anleitung (installiert nur einmal je Rechner) oder direkte Remote-Installation über die Admin-Freigabe `C$` (Anmeldedaten nötig; kopiert und startet den Dienst).

Agents im Netzwerk: „Agents suchen“ prüft das aktuelle Subnetz oder einen manuellen Bereich (von/bis, bis 4096 Adressen, auch über mehrere /24-Netze) auf dem Port des Agents (der API-Key wird nur an diesen Port gesendet). Die Tabelle zeigt Status, Hostname, IP-Adresse, Version, Modus, Läuft ab und Letzte Erfassung. Aktionen für die markierten Agents (ohne Markierung alle): „Inventar abrufen“ (lädt die letzte Erfassung in den Abrufordner), „Jetzt erfassen“ (sofortige Neuerfassung, Ergebnis danach per Abruf), „Prüfen“ (Erreichbarkeit) und „Deinstallieren“.

Ablage im Agent (ab 1.5, `InventoryStore`): Neben der geplanten Erfassung speichern auch die Erfassung beim Dienststart und `POST /collect` die Datei im Ablageort – eine erste Datei liegt also gleich nach der Installation vor, verpasste Termine holt der nächste Start nach. Dateiname einheitlich `<Kunden-Nr>_<Rechnername>.json` (`InventoryResult.FileName()`, auch beim Abruf); eine Datei im alten Schema `<Rechnername>.json` entfernt der Agent. Ist der Ablageort nicht erreichbar, liegt die Datei verschlüsselt unter `C:\ProgramData\DATEXT\Agent\Ausstehend` und wird stündlich sowie 30 s nach jeder Netzwerkänderung nachgereicht. `/status` meldet `lastSaved`, `savePending` und `saveError`; die Liste zeigt sie in der Spalte „Im Ablageort“.

Selbstaktualisierung (ab 1.5, `AgentUpdater`): Das Installationsskript schreibt seinen Ordner nach `update-source.txt` im Programmordner (nicht bei RunOnce); der Agent prüft ihn spätestens eine Stunde nach dem Start, dann täglich. Alternativ `POST /update` mit der EXE im Body (Schaltfläche „Aktualisieren“, nur Agents ab 1.5 mit älterer Version als die eingebettete). Angenommen wird nur eine höhere Version, gültig signiert

(WinVerifyTrust ohne Online-Sperrprüfung) und mit demselben Zertifikats-Subject wie die laufende EXE; ein unsignierter Agent aktualisiert sich nie. Die Datei wird vor der Prüfung nach `C:\Program Files\DATEXTAgent\update` kopiert. Den Austausch macht eine Kopie der alten EXE (`--apply-update`): Dienst stoppen, alte EXE sichern, neue einsetzen, starten; läuft der Dienst nach 20 s nicht, wird die alte wiederhergestellt. Protokoll `C:\ProgramData\DATEXT\Agent\update.log`. Das Installationsskript vergleicht Versionen (`-ge`) statt Gleichheit und stuft einen selbst aktualisierten Agent nicht zurück. `/status` meldet `updateSource` und `updateError` (Tooltip der Versionsspalte).

Quellen der Auswertung (`.Sources`): `InventorySource` je Ordner – Ablageort jedes Kundenprofils mit UNC-Pfad (gleicher Pfad mehrerer Kunden = eine Quelle; lokale Pfade liegen auf den Zielrechnern und entfallen), Abrufordner (fällt er mit einem Ablageort zusammen, nur dieser mit Vermerk) und eine Quelle „Importiert“ für Dateien von Hand. Neu angehakt sind Abrufordner und Ablageort des aktuellen Kunden; abgeglichen wird beim Laden, bei Kundenwechsel und beim Wechsel auf den Reiter. Eingelesen wird im Hintergrund (ein nicht erreichbarer Server blockiert sonst bis ~20 s), danach je Quelle ein `FileSystemWatcher` (Created/Changed/Renamed/Deleted; bei `Error` gilt die Quelle als nicht erreichbar). Abhaken stoppt den Wächter und verwirft die Kopien; beim Entladen des Dialogs werden die Wächter gestoppt und beim nächsten Laden neu eingelesen. Je Quelle und Gerät (Kunden-Nr + Hostname) gilt der neueste Stand (`InventoryCopy`), angezeigt wird je Gerät der neueste aller angehakten Quellen (`RebuildInventory`; die Zeile wird je Kopie einmal gebaut, die Markierung bleibt). Spalte „Quelle“: Quelle des angezeigten Stands, weitere in Klammern, Tooltip mit allen Kopien. **Ablage veraltet:** Inventar ab Agent 1.5, angezeigter Stand nicht aus dem Ablageort, Ablageort des Kunden angehakt und erreichbar, dort aber keine Kopie („fehlt im Ablageort“) oder eine mehr als 24 h ältere – dann Warnfarbe und Kachel „Ablage veraltet“ (nur wenn ein Ablageort eingelesen ist).

Import & Auswertung: „Ordner importieren …“ und „Dateien …“ laden verschlüsselte oder unverschlüsselte Inventardateien in die Quelle „Importiert“; ein Ordner, der eine Quelle ist, wird stattdessen als diese angehakt und neu eingelesen. „Inventar abrufen“ ordnet die Ergebnisse der Quelle des Abrufordners zu. **Abrufordner:** `SetDefaultFetchFolder` (in `ApplyProfile`, also bei jedem Kunden- bzw. Profilwechsel) setzt den UNC-Ablageort des Profils, sonst Dokumente\DATEXT\Inventare. Vor dem Abruf prüft `FetchFolderProblem` (Existenz, Testdatei `.abruftest-<PC>.tmp`); scheitert die automatische Vorgabe, weicht der Abruf mit Hinweis auf den lokalen Ordner aus, ein selbst gewählter Ordner bricht mit Fehler ab. Der lokale Ordner erscheint, falls vorhanden, als nicht angehakte Quelle „Abrufordner lokal“. Die Tabelle zeigt Rechner, Kunde, Betriebssystem, Build, CPU, RAM, Datenträger, Programme, Letztes Update, Erfasst und die Bewertung (P = Probleme, H = Hinweise, gleiche Bewertung wie die lokale Inventarisierung). Suchfelder für Rechner/Kunde/OS/CPU und für Software über alle Geräte (z. B. „Office“); Doppelklick öffnet das Detailfenster (mit Vergleich zu einem früheren Stand). Export: Übersicht als CSV, Inventare gespeichert (verschlüsselt) oder als JSON (unverschlüsselt, für Import) sowie der Detailbericht als PDF.

© 2026 DATEXT

11. Dienste & Hintergrundkomponenten

NetworkDiagnosticsService

Datei: `Services/NetworkDiagnosticsService.cs`

Zentrale Serviceklasse für Netzwerkoperationen. Wird per Dependency Injection (Konstruktor-Parameter) in alle Netzwerk-Dialoge eingebracht. Kapselt Hilfsmethoden für ICMP-Ping, DNS-Auflösung und Netzwerkadapter-Abfragen.

NetworkOverviewService

Datei: `Services/NetworkOverviewService.cs`

Aggregiert Systeminformationen für den Übersichtsdialog: primärer Netzwerkadapter, Gateway, DNS, externe IP-Adresse (via HTTP), Systeminformationen.

DnsLookupService

Datei: `Services/DnsLookupService.cs`

Dateien: `Services/DnsLookupService.cs` sowie `.Checks.cs`, `.Dnssec.cs`, `.Servfail.cs`, `.Services.cs` (Partial-Klasse)

Kapselt die Abfragen der IP & DNS Analyse über das `DnsClient`-NuGet-Paket und gibt ein typisiertes `DnsLookupResult` (Modell in `Models/DnsLookupResult.cs`) zurück. Ablauf: Phase 1 (Records, Reverse, DNSSEC, CAA, Mail-Einträge, Propagation), Phase 2 (Verweisprüfung, dann nacheinander erweiterte Prüfungen, autoritative Server, Wildcard, Diensterkennung), danach Vergleich mit der autoritativen Antwort, TTL-Hinweise und – bei SERVFAIL – die Diagnose.

- **Checks:** Direktabfrage der autoritativen Server ohne Rekursion (`DirectClient`, `QueryAuthServerAsync`), Verweisprüfung (CNAME-Kette, MX-Ziele, `TakeoverServices`), Team-Cymru-Abfragen für ASN, Netz und Registry
- **Dnssec:** `QueryLargeAsync` (UDP ohne Fallback, dann TCP mit 3 s Timeout; gesperrtes TCP/53 wird pro Resolver 2 Minuten gemerkt), Schlüssel-ID nach RFC 4034 Anhang B, DS-Prüfsummen, Gültigkeitsfenster, TTL-Hinweise, Wildcard-Prüfung
- **Servfail:** Iterative Auflösung ab den 13 Root-Servern mit Glue und DS aus den Referrals; kryptografische Prüfung von RRSIG (RSA, ECDSA P-256/P-384)
- **Services:** Mustertabellen für Mail- und Versandanbieter, CNAME-Proben und TXT-Bestätigungen (`DoServiceDetectionAsync`)

MailDomainAnalyzer

Datei: `Services/MailDomainAnalyzer.cs`

Gemeinsame DNS-Auswertung einer Mail-Domain für E-Mail Domain-Analyse, E-Mail-Check und Header-Analyse. `AnalyzeAsync(domain, …, senderIp, parts)` liefert ein Berichtsobjekt mit SPF, DKIM, DMARC, MX, DNS-Sicht, MTA-STS, TLS-RPT, DANE, BIMI und Reputation; `MailDomainParts` wählt die Teilprüfungen (z. B. nur SPF mit Absender-IP für die Header-Analyse).

MailSecurityAnalysisService

Datei: `Services/MailSecurityAnalysisService.cs`

Prüfungen des E-Mail-Checks, die vom eigenen Rechner abhängen: DKIM, DANE, PTR, TLS-Bewertung nach BSI TR-02102, MX-Test auf Port 25, IMAP/POP3. SPF und DMARC kommen aus dem `MailDomainAnalyzer`.

© 2026 DATEXT

GraphMailTestService

Datei: `Services/GraphMailTestService.cs`

Testversand über Microsoft Graph für Microsoft 365: Anmeldung per MSAL (`Microsoft.Identity.Client`) im Browser, Versand über `POST /me/sendMail`. Enthält die gemeinsame Signatur `Disclaimer`, die auch der SMTP-Test nutzt; die Client-ID der DATEXT-App-Registrierung steht in dieser Datei.

DomainAnalysisService

Datei: `Services/DomainAnalysisService.cs`

Nur noch für KIM-Domains: Anbieter-Erkennung und optionale TI-Analyse (TI-DNS, Konnektor). Für alle anderen Domains verweist der Dienst auf den `MailDomainAnalyzer`.

FingerbankApiService

Datei: `Services/FingerbankApiService.cs`

Optionale Integration der Fingerbank-API zur Geräteklassifizierung im Geräte-Scan. API-Schlüssel wird über `FingerbankConfigDialog` konfiguriert.

LocalDeviceIdentificationService

Datei: `Services/LocalDeviceIdentificationService.cs`

Identifiziert Gerätetypen im lokalen Netzwerk anhand von MAC-Vendor, Banner-Grabbing-Ergebnissen und Hostname-Mustern. Wird im Geräte-Scan eingesetzt.

Etl2PcapngService

Datei: `Services/Etl2PcapngService.cs`

Konvertiert Windows ETL-Trace-Dateien (PKTMON, NETSH) in das PCAPNG-Format für Wireshark. Gibt Status-Meldungen über ein `LogMessage`-Event aus.

SettingsService

Datei: `Services/SettingsService.cs`

Persistiert Anwendungseinstellungen (z. B. Fingerbank API-Key, zuletzt verwendete IP-Bereiche).

ExportService

Datei: `Services/ExportService.cs`

Generiert CSV- und JSON-Exporte für verschiedene Dialoge. **PDF-Berichte** aller Dialoge entstehen über `InventoryPdfExporter.ExportReport` bzw. `ExportLogReport` (Dialogs/Inventory/): DATEXT-Kopf mit Titelblock, zweispaltige Werte, Tabellen und farbige Bewertungsblöcke, auf Basis von `PdfSharp`. Dateinamen vergibt `ExportFile.Name`.

SmtpTestService

Datei: `Services/SmtpTestService.cs`

Kapselt die SMTP-Verbindungslogik für den E-Mail-Check. Nutzt MailKit für Verbindungsaufbau, Authentifizierung und TLS-Zertifikats-Inspektion.

© 2026 DATEXT

SnmpService

Datei: `Services/SnmpService.cs`

SNMP-Abfragen via `Lextm.SharpSnmpLib`. Wird im Geräte-Scan für SNMP-fähige Geräte (Router, Switches) genutzt.

12. NuGet-Pakete & Abhängigkeiten

Paket	Version	Verwendung
MailKit	4.18.1	SMTP-Verbindungstest, IMAP-Prüfung
MimeKit	4.18.1	E-Mail-Parsing, MIME-Typen
DnsClient	1.8.0	DNS-Abfragen aller Record-Typen, DNSSEC-Records, Direktabfragen
Microsoft.Identity.Client	4.90.1	MSAL-Anmeldung für den Graph-Testversand (Microsoft 365)
System.Security.Cryptography.ProtectedData	10.0.12	DPAPI für gespeicherte Agent-API-Keys
System.Data.SqlClient	4.9.1	MS-SQL-Analyse (Umstellung auf Microsoft.Data.SqlClient geplant)
Lextm.SharpSnmpLib	12.5.7	SNMP v1/v2c/v3-Abfragen
PdfSharp	6.2.4	PDF-Export
Newtonsoft.Json	13.0.4	JSON-Verarbeitung (Speedtest, API)
TaskScheduler	2.12.2	Autorun-Analyse (geplante Aufgaben)
System.Management	10.0.12	WMI-Abfragen (Hardware, TPM, etc.)
System.ServiceProcess.ServiceController	10.0.12	Windows-Dienste lesen/steuern
System.Text.Encoding.CodePages	10.0.12	Codepage 850 (route print, netstat)

© 2026 DATEXT

13. Navigation & Dialog-Kommunikation

NavigateToSidebarRequested-Event

Mehrere Dialoge lösen eine Navigation zu einem anderen Dialog aus. Das Kommunikationsprotokoll verwendet ein typisiertes Event:

```
public event EventHandler<string>? NavigateToSidebarRequested;
```

Das String-Argument folgt dem Format `"aktion:Zieladresse"` (Aktion in Kleinbuchstaben), z. B.:

Format	Auslöser	Ziel
`ping:192.168.1.1`	Gateway anpingen (NetworkCards), Schnellaktion der IP & DNS Analyse	Ping-Dialog
`dns:google.com`	Abfrage aus Geräte-Scan, Netzwerkkarten oder Übersicht	IP & DNS Analyse
`traceroute:8.8.8.8`	Route verfolgen	Traceroute-Dialog
`portscan:192.168.1.1`	Ports scannen	Port-Scan-Dialog
`emailanalysis:example.de`	Button „Domain-Analyse“ der IP & DNS Analyse	E-Mail Domain-Analyse
`ad:COMPUTERNAME`	AD-Suche	Active-Directory-Dialog

Wichtig: Das Event muss beim Erstellen des Dialogs explizit verdrahtet werden:

```
nameof(rbNetworkCards) => GetOrCreate(ref _networkCardsDialog, () => {
    var d = new NetworkCardsDialog(_diagnosticsService);
    d.NavigateToSidebarRequested += OnNavigateToSidebarRequested;
    return d;
}),
```

SetHostname-Methode

Dialoge, die einen Hostnamen/eine IP-Adresse als Eingabe akzeptieren, implementieren `SetHostname(string)`. Diese Methode setzt das Eingabefeld und startet die Aktion direkt:

```
public void SetHostname(string hostname)
{
    txtHostname.Text = hostname;
    if (!string.IsNullOrWhiteSpace(hostname))
        BtnPing_Click(this, new RoutedEventArgs());
}
```

Lazy Loading & Caching

Alle Dialoge werden erst bei erster Aktivierung instanziiert (`??=`-Operator oder `GetOrCreate()`). Einmal erstellt, werden sie im `MainWindow` als Felder gecacht und bei wiederholter Navigation wiederverwendet – ohne den Zustand zu verlieren.

© 2026 DATEXT

14. Deployment & Veröffentlichung

Build-Konfiguration

Die Anwendung wird als vollständig selbstständige Einzeldatei veröffentlicht:

```
<PublishSingleFile>true</PublishSingleFile>
<SelfContained>true</SelfContained>
<RuntimeIdentifier>win-x64</RuntimeIdentifier>
<EnableCompressionInSingleFile>true</EnableCompressionInSingleFile>
<IncludeNativeLibrariesForSelfExtract>true</IncludeNativeLibrariesForSelfExtract>
<IncludeAllContentForSelfExtract>true</IncludeAllContentForSelfExtract>
```

Publish-Profile

Folgende Veröffentlichungsprofile sind hinterlegt:

- **win-x64 Export** – Standard-Windows 64-Bit-Ziel

MSI-Installation (Setup)

Neben der portablen Single-File-EXE gibt es eine MSI (`DATEXT-Diagnostics-Setup.msi`, x64, per-machine) aus dem Projekt `Setup\DATEXT.Diagnostics.Setup` (WiX Toolset **v5.0.2**, bewusst nicht v6/v7 wegen der kostenpflichtigen Wartungsgebühr ab v6).

- Installiert nach `Programme\DATEXT GmbH\DATEXT Diagnostics`, legt Startmenü- und Desktop-Verknüpfung an und registriert sich in „Programme und Features“; Versions-Upgrades über eine feste `UpgradeCode`. Oberfläche WixUI_InstallDir auf Deutsch (Lizenztext, wählbarer Zielordner).
- Die MSI installiert einen self-contained **mehrdateiigen** Publish (`PublishSingleFile=false`, Ordner `publish-msi`, rund 500 Dateien), damit zur Laufzeit nichts nach `%TEMP%\.net` entpackt werden muss.
- Das Target `SignMainExecutable` signiert `DATEXT-Diagnostics.exe` zwischen Publish und Paketbau (Zertifikat per SHA1-Thumbprint aus dem Windows-Zertifikatspeicher, kein PFX). Ohne Thumbprint bricht der Build kontrolliert ab.
- Aufruf: `dotnet build Setup\DATEXT.Diagnostics.Setup\DATEXT.Diagnostics.Setup.wixproj -c Release -p:CodeSignThumbprint=<SHA1>`
- `ProductVersion` in der `.wixproj` muss bei jedem Release der `<Version>` in `DATEXT.Diagnostics.csproj` angeglichen werden. Die Lösungsdatei baut ohne Thumbprint nicht durch; für Entwicklungsbuilds nur `DATEXT.Diagnostics.csproj` bauen.

Eingebetteter Agent

`Resources\DATEXTAgent.exe` ist der signierte Inventory Agent (aktuell 1.1.4). Er wird nicht bei jedem Build neu erzeugt, sondern aus dem Publish des Projekts `DATEXTAgent` mit dem EV-Zertifikat übernommen; ein unsignierter Neubau darf ihn nicht ersetzen.

Eingebettete Ressourcen

Ressource	Format	Einbettung
MAC-Vendor-Datenbank	JSON	CopyToOutputDirectory
ISP/Provider-Daten	JSON	EmbeddedResource
App-Icon	ICO	ApplicationIcon
Firmenlogo	JPG	Resource

© 2026 DATEXT

Administrator-Rechte

Folgende Features erfordern erhöhte Rechte (app.manifest: `requireAdministrator` oder Laufzeitprüfung):

- DHCP-Discovery (UDP-Broadcast auf Port 67)
- PKTMON Capture
- NETSH Trace
- Troubleshooting (Windows-Update-Reparatur, Netzwerk-Reset, DISM /RestoreHealth, SFC)
- UEFI-Zertifikate lesen (teilweise)
- Lokale Inventarisierung (vollständige Erfassung, Gruppenrichtlinien mit `gpresult`)
- Installation und Verteilung des Inventory Agents

Dokument erstellt aus dem Quellcode des Projekts DATEXT.Diagnostics v0.99.12.11 – November 2026

© 2026 DATEXT